

빅데이터 분석 기사 시험 소개

with  python™

배 포 금 지

01. 실기시험 출제 유형

(2023년 11월 기준)

빅데이터분석기사 실기시험의 출제유형이 아래와 같이 변경됨을 안내합니다.

변경내용 : 기존 단답형 10문제를 작업형 신규 유형인 '작업형 제3유형'으로 대체

변경시기 : 제6회 빅데이터분석기사 실기시험(2023.06.24)부터 적용

변경 전 (~제5회)			⇒	변경 후 (제6회~)		
유형	문제수	배점		유형	문제수	배점
작업형 제1유형	3문제	30점 (각10점)		작업형 제1유형	3문제	30점 (각10점)
작업형 제2유형	1문제	40점		작업형 제2유형	1문제	40점
<u>단답형</u>	<u>10문제</u>	<u>30점 (각3점)</u>		<u>작업형 제3유형</u>	<u>2문제</u>	<u>30점 (각15점)</u>
합계	<u>14문제</u>	100점		합계	<u>6문제</u>	100점

◆ 1유형

TEST 체험하기

0 / 4 제한 시간 없음

i ⚙

문제정보

🔍 100% 🔍

제1유형

mtcars 데이터셋(data/mtcars.csv)의 qsec 컬럼을 최소최대 척도(Min-Max Scale)로 변환한 후 0.5보다 큰 값을 가지는 레코드 수를 구하시오.

* dataset 위치 : data/mtcars.csv

Python3 data/mtcars.csv Python3 초기화 저장 실행 테스트케이스 추가 테스트 제출

1 # 출력을 원할 경우 print() 함수 활용

2 # 예시) print(df.head())

3

4 # getcwd(), chdir() 등 작업·폴더 설정 불필요

5 # 파일·경로 상 내부·드라이브·경로(C: 등) 접근 불가

6

7 # 데이터·파일 읽기 예제

8 import pandas as pd

9 a = pd.read_csv('data/mtcars.csv', index_col=0)

10

11 # 사용자 코딩

12

13 # 답안 제출 예시

14 # print(평균변수값)

15

실행결과 제출결과 테스트케이스

정지

궁금한 건 채팅으로 문의하세요
● 보통 수십 분 내 답변

빅데이터 분석기사 상시 체험

1/4

다음 >

- 4 -

공동: 정지훈 강사 / jhjung@dschloe.com / 01072072163 Copyright © 디에스클로에. All Rights Reserved. 무단 전재 및 배포 금지

◆ 2유형

TEST 채점하기

0 / 4 제한 시간 없음

문제정보

제2유형

아래는 백화점 고객의 1년 간 구매 데이터이다.

(가) 제공 데이터 목록

1. y_train.csv : 고객의 성별 데이터 (학습용), CSV 형식의 파일

2. X_train.csv, X_test.csv : 고객의 상품구매 속성 (학습용 및 평가용), CSV 형식의 파일

(나) 데이터 형식 및 내용

1. y_train.csv (3,500명 데이터)

	cust_id	gender
0	0	0
1	1	0
2	2	1
3	3	1
4	4	0
5	5	0

Python3

data/X_test.csv

data/X_train.csv

data/y_train.csv

Python3

초기화

저장

실행

제출

```
1 # 출력을 원하실 경우 print() 함수 활용
2 # 예시) print(df.head())
3
4 # getcwd(), chdir() 등 작업 폴더 설정 불필요
5 # 파일 경로 상 내부 드라이브 경로(C: 등) 접근 불가
6
7 # 데이터 파일 읽기 예제
8 import pandas as pd
9 X_test = pd.read_csv("data/X_test.csv")
10 X_train = pd.read_csv("data/X_train.csv")
11 y_train = pd.read_csv("data/y_train.csv")
12
13 # 사용자 코딩
14
15 # 답안 제출 참고
16 # 아래 코드 예측변수와 수험번호를 개인별로 변경하여 활용
17 # pd.DataFrame({'cust_id': X_test.cust_id, 'gender': pred}).to_csv('003000000.csv', index=False)
18
```

실행결과

제출결과

02. 유형별 예시

(2023년 11월 기준)

◆ 3유형

TEST 체험하기

0 / 4 제한 시간 없음

문제정보

제3유형 (풀이용, 제출없음)

주어진 데이터(data/blood_pressure.csv)에는 고혈압 환자 120명의 치료 전후의 혈압이 저장되어 있다. 해당 치료가 효과가 있는지 (즉, 치료 후의 혈압이 감소했는지) 쌍체표본 t-검정(paired t-test)을 통해 답하고자 한다. 가설은 아래와 같다.

$$\mu_d: (\text{치료 후 혈압} - \text{치료 전 혈압}) \text{의 평균}$$
$$H_0: \mu_d \geq 0$$
$$H_1: \mu_d < 0$$

- bp_before : 치료 전 혈압
- bp_after : 치료 후 혈압

① μ_d 의 표본평균을 입력하십시오. (반올림하여 소수 둘째자리까지 계산)
② 위의 가설을 검정하기 위한 검정통계량을 입력하십시오. (반올림하여 소수 넷째자리까지 계산)
③ 위의 통계량에 대한 p-값을 구하여 입력하십시오. (반올림하여 소수 넷째자리까지 계산)

Python3

data/blood_pressure.csv

Python3

초기화

저장

실행

테스트케이스 추가

테스트

제출

```
1 # 출력을 원할 경우 print() 함수 활용
2 # 예시 print(df.head())
3
4 # getcwd(), chdir() 등 작업 폴더 설정 불필요
5 # 파일 경로 상 내부 드라이브 경로(C: 등) 접근 불가
6
7 import pandas as pd
8
9 a = pd.read_csv('data/blood_pressure.csv', index_col=0)
10
11
12 # 사용자 코딩
13
14 # 해당 화면에서는 제출하지 않으며, 문제 풀이 후 답안제출에서 결과값 제출
```

실행결과

제출결과

테스트케이스

정지

goor

빅데이터 분석기사 상시 체험

< 이전

3/4

다음 >

03. 응시 자격 기준

(2023년 11월 기준)

•다음 중 하나에 해당하는 사람

1. 대학졸업자등 또는 졸업예정자 (전공 무관)
2. 3년제 전문대학 졸업자등으로서 졸업 후 1년 이상 직장경력이 있는 사람 (전공, 직무분야 무관)
3. 2년제 전문대학 졸업자등으로서 졸업 후 2년 이상 직장경력이 있는 사람 (전공, 직무분야 무관)
4. 기사 등급 이상의 자격을 취득한 사람 (종목 무관)
5. 기사 수준 기술훈련과정 이수자 또는 그 이수예정자 (종목 무관)
6. 산업기사 등급 이상의 자격을 취득한 후 1년 이상 직장경력이 있는 사람 (종목, 직무분야 무관)
7. 산업기사 수준 기술훈련과정 이수자로서 이수 후 2년 이상 직장경력이 있는 사람 (종목, 직무분야 무관)
8. 기능사 등급 이상의 자격을 취득한 후 3년 이상 직장경력이 있는 사람 (종목, 직무분야 무관)
9. 4년 이상 직장경력이 있는 사람 (직무분야 무관)

※ 졸업증명서 및 경력증명서 제출 필요

파이썬 기초 문법

with  python™

◆ 숫자형(Number)

항목	표현 예시
정수 (int)	1, 2, 3, 123, 1345, 0, -123, -1, -2
실수 (float)	12.34, 456.189, 1.2e+5*, 1.2e-5**

☒ Scientific Notation
* $1.2e+5 = 1.2 \times 10^5 = 120000$
** $1.2e-5 = 1.2 \times 10^{-5} = 0.000012$

◆ 문자열(String)

주요 기능	정의
더하기	더하기 순서대로 문자열을 하나로 연결한다는 것을 의미
곱하기	문자열을 반복해서 연결하는 것을 의미
인덱싱	특정 위치에 있는 문자만 지정하여 일부를 추출함. 인덱스번호는 0번째부터 시작함
슬라이싱	특정 위치에 있는 범위를 지정하여 전체 또는 1개 이상의 문자를 추출함

02. 문자열 주요 메서드

(2023년 11월 기준)

◆ 메서드, a는 임의의 문자열이 저장된 객체

주요 메서드	메서드 설명
<code>a.count('p')</code>	특정 문자 ('p')가 몇 개가 있는지 확인
<code>a.find('p')</code>	특정 문자 ('p')가 첫번째로 등장한 위치(인덱스) 번호를 확인
<code>a.upper()</code>	영어 문자를 대문자로 변환
<code>a.lower()</code>	영어 문자를 소문자로 변환
<code>a.lstrip()</code>	문자열 왼쪽에 있는 공백 제거
<code>a.rstrip()</code>	문자열 오른쪽에 있는 공백 제거
<code>a.strip()</code>	문자열 양쪽에 있는 공백 제거
<code>a.replace("x", "y")</code>	문자열에 포함된 x를 y로 변경
<code>b = ', '.join(a)</code>	기존 문자열(a)에 특정 문자(예: ,)를 문자열 사이마다 삽입할 수 있음
<code>a.split(조건)</code>	문자열을 특정 조건에 따라 나눠서 다수의 문자열로 구성된 리스트 자료형으로 변경

◆ 리스트는 숫자형, 문자열 자료형을 하나의 집합으로 구성할 수 있음

```
1 : []  
2 : [2,4,6,8,10]  
3 : ['빅데이터', '분석', '기사']  
4 : [1, 2, 3, ['빅데이터', '분석', '기사']] # 중첩리스트(Nested List)  
5 : ['빅데이터', '분석', '기사', 1, 2, 3]
```

◆ 문자열과 마찬가지로 더하기, 곱하기 같은 사칙연산을 활용해서 리스트를 합치거나 반복 가능

```
1 : a = [1, 2, 3]  
2 : b = [4, 5, 6]  
3 : c = a + b  
4 : d = c * 3  
5 : print(d)
```

- ◆ 문자열과 마찬가지로 인덱싱과 슬라이싱 가능
- ◆ 중첩된 리스트에 대해서도 인덱싱과 슬라이싱 가능

```
1 : a = [1, 2, 3, ["a", "b", "c", "d"]]  
2 : print(a)  
3 : print(a[3][0]) # 중첩된 리스트 인덱싱 및 슬라이싱
```

```
[1, 2, 3, ['a', 'b', 'c', 'd']]  
a
```

◆ a는 임의의 리스트가 저장된 객체

주요 메서드	메서드 설명
a.append(x)	리스트의 맨 마지막 요소 위치에 x라는 요소를 추가한다.
a.sort()	리스트의 요소들을 정렬하는 함수가 있다.
a.index(x)	리스트 내의 x값의 위치에 해당하는 인덱스 값을 반환한다.
a.insert(x, y)	x값은 인덱스 번호, y값은 특정 값으로 x 인덱스 위치에 y 값을 추가한다.
a.remove(x)	x는 특정값을 의미하며, 리스트에서 x 값을 제거한다.
a.pop(x)	x는 인덱스 번호를 의미, 리스트에서 해당 인덱스의 값을 반환하며, 리스트에서 제거한다.
a.count(x)	특정요소 x의 개수를 계산한다.

04. 튜플(Tuple) 자료형

(2023년 11월 기준)

◆ 튜플과 리스트는 비슷한 역할을 수행함. 대괄호가 아닌 소괄호를 사용해야 함.

```
1 : (1, 2, 3)
2 : (1, )
3 : 1, 2, 3
4 : (1, 2, 3, ('빅데이터', '분석', '기사')) # 중첩튜플(Nested Tuple)
5 : ('빅데이터', '분석', '기사', 1, 2, 3)
```

◆ 리스트와 마찬가지로 더하기, 곱하기 같은 사칙연산을 활용해서 튜플을 합치거나 반복 가능

◆ 인덱싱과 슬라이싱 가능

◆ 리스트와의 가장 큰 차이점

- 리스트 : 요소의 생성, 삭제, 수정 가능
- 튜플 : **요소 변경 불가능**

05. 딕셔너리(Dictionary 자료형)

(2023년 11월 기준)

◆ 키(Key)와 값(Value)으로 이루어진 자료형

```
1 : a = {'name' : 'evan', 'age' : 30, 'birth' : [4, 30]}
```

◆ Value 값을 구하기 위해서는 Key를 활용해야 함.

```
1 : a = {'name' : 'evan', 'age' : 30, 'birth' : [4, 30]}
2 : print(a['name'])
3 : print(a['birth'])
```

```
evan
[4, 30]
```

05. 딕셔너리(Dictionary 자료형) - 메서드

(2023년 11월 기준)

◆ 주요 메서드는 다음과 같음, a는 임의의 딕셔너리로 저장된 객체

주요 메서드	메서드 설명
a.keys()	딕셔너리의 키(Key)의 리스트를 추출할 수 있음
a.values()	딕셔너리의 값(Value)의 리스트를 추출할 수 있음
a.items()	딕셔너리의 Key, Value를 튜플 구조로 묶고, 리스트로 추출할 수 있음
a.get(x, y)	딕셔너리의 x의 키가 존재하지 않을 때 y값 반환하도록 처리

◆ If문 : 조건문1을 테스트하여 참이면 if문, 아니면 elif 조건문2를 테스트하여 참이면 코드

```
1  :  if 조건문1:
2  :      코드 실행 1 # 들여쓰기는 공백(Spacebar) 또는 탭(Tab) 사용
3  :      코드 실행 2
4  :  elif 조건문2:
5  :      코드 실행 1
5  :      코드 실행 2
6  :  else:
7  :      코드 실행 1
8  :      코드 실행 2
9  :  A = [1, 2, 3]
10 :  ...
```

◆ 조건문 작성 방법

조건문 표현 방법	조건문 의미
$x < y$	x가 y보다 작다면
$x > y$	x가 y보다 크다면
$x == y$	x와 y가 같다면
$x != y$	x와 y가 같지 않다면
$x >= y$	x가 y보다 크거나 같다면
$x <= y$	x가 y보다 작거나 같다면

06. 파이썬 제어문

(2023년 11월 기준)

◆ 조건문 비교 연산자 (AND)

X	Y	Result
True	True	True
True	False	False
False	True	False
False	False	False

◆ 조건문 비교 연산자 (OR)

X	Y	Result
True	True	True
True	False	True
False	True	True
False	False	False

◆ not x : x가 거짓이면 참이다

◆ while 반복문 : 조건문이 참인 동안에 while 아래의 문장을 반복해서 수행함

```
1  : a = 0
2  :
3  : while a < 3:
4  :     print(f'현재 a값은 {a} 입니다.')
5  :     a = a + 1
6  :
7  : print("종료")
```

```
현재 a값은 0 입니다.
현재 a값은 1 입니다.
현재 a값은 2 입니다.
종료
```

◆ for 반복문 : 리스트, 문자열, 튜플 등 0번째 인덱스부터 마지막 인덱스까지 차례로 변수에 대입되어 명령문이 실행

```
1 : numbers = [100, 200, 300]
2 :
3 : for num in numbers:
4 :     print(num)
5 :
6 : print("종료")
```

```
100
200
300
종료
```

07. 사용자 정의 함수

(2023년 11월 기준)

◆ def문 : 함수 이름을 임의로 만들고, 이름 뒤 괄호 안의 매개변수는 입력으로 전달되는 값을 받는 변수

```
1  :  def 함수이름(매개변수1, 매개변수2, ...)  
2  :      코드 1  
3  :      코드 2  
4  :      ...  
5  :      코드 N  
6  :      return 결과값
```

파이썬 데이터 분석 라이브러리

with  python™

◆ 응시환경에서 다음 명령어를 이용하여 설치된 패키지 확인 가능

```
1 : import pkg_resources
2 : import pandas
3 : OutputDataSet = pandas.DataFrame(sorted([(i.key, i.version) for i in pkg_resources.working_set]))
4 : print(OutputDataSet)
```

1	import pkg_resources	
2	import pandas	
3	OutputDataSet = pandas.DataFrame(sorted([(i.key, i.version) for i in pkg_resources.working_set]))	
4	print(OutputDataSet)	
실행결과	제출결과	테스트케이스
3	chardet	3.0.4
4	cryptography	2.1.4
5	cycler	0.10.0
6	cython	0.29.24
7	distlib	0.3.2
8	idna	2.6
9	joblib	1.0.1
10	keyring	10.6.0
11	keyrings.alt	3.0
12	kiwisolver	1.3.1
13	lightgbm	3.3.2
14	matplotlib	3.4.2
15	numpy	1.21.1
16	packaging	23.0
17	pandas	1.4.2

◆ 데이터과학 주요 라이브러리

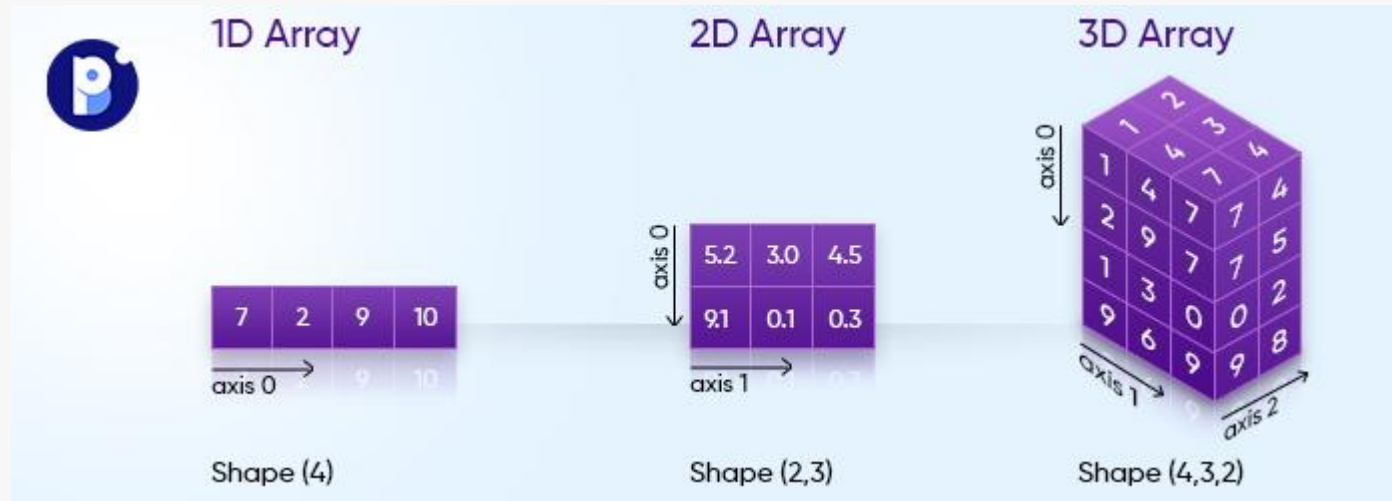
라이브러리명	버전	라이브러리 주요 설명
lightgbm	3.3.2	MS에서 만든 Gradient Boosting에 기반한 머신러닝 모형 라이브러리
matplotlib	3.4.2	이미지 및 데이터 시각화 라이브러리
numpy	1.21.1	선형대수 기반의 프로그램을 쉽게 만들 수 있도록 도와주는 라이브러리
pandas	1.4.2	데이터 처리, 가공, 집계 등을 쉽게 할 수 있도록 도와주는 라이브러리
scikit-learn	0.24.2	Python 머신러닝의 핵심라이브러리, 다양한 알고리즘을 지원함
scipy	1.7.0	과학, 분석, 특히 다양한 통계모듈을 지원하는 라이브러리
statsmodels	0.13.5	다양한 통계 모델링 및 계량경제 분석 도구 제공하는 라이브러리
xgboost	1.4.2	Gradient Boosting에 기반한 머신러닝 라이브러리

02. numpy

(2023년 11월 기준)

◆ numpy의 기반 데이터 타입은 다차원 배열(ndarray)

- ✓ 1차원 배열
- ✓ 2차원 배열
- ✓ 3차원 배열



출처 : <https://www.programsbuzz.com/article/numpy-array-axis>

02. numpy

(2023년 11월 기준)

◆ numpy 배열 인덱싱

	COL=0	COL=1	COL=2
ROW=0	(0, 0) 1	(0, 1) 2	(0, 2) 3
ROW=1	(1, 0) 4	(1, 1) 5	(1, 2) 6
ROW=2	(2, 0) 7	(2, 1) 8	(2, 2) 9

`array[0, 1]`

02. numpy

(2023년 11월 기준)

◆ numpy 배열 슬라이싱

	COL=0	COL=1	COL=2
ROW=0	(0, 0) 1	(0, 1) 2	(0, 2) 3
ROW=1	(1, 0) 4	(1, 1) 5	(1, 2) 6
ROW=2	(2, 0) 7	(2, 1) 8	(2, 2) 9

`array1[0:2, 0:2]`

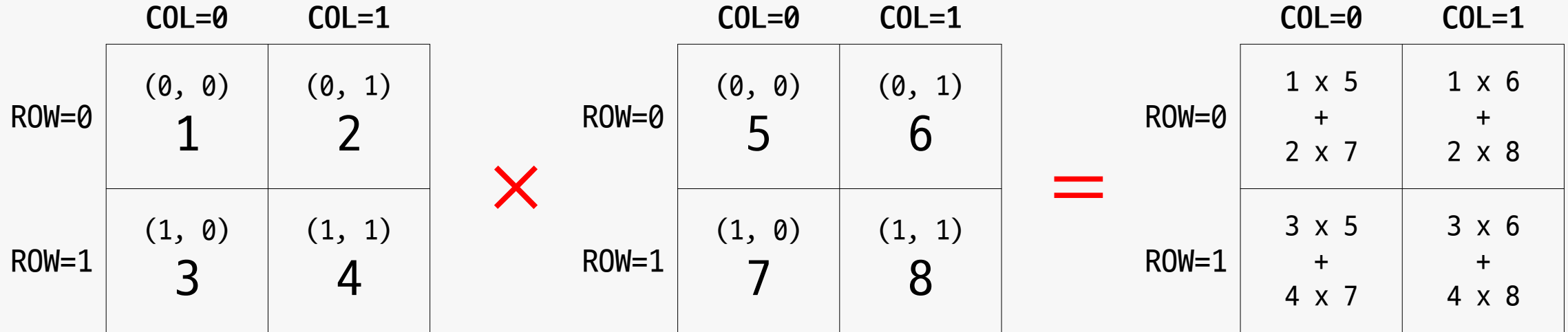
	COL=0	COL=1	COL=2
ROW=0	(0, 0) 1	(0, 1) 2	(0, 2) 3
ROW=1	(1, 0) 4	(1, 1) 5	(1, 2) 6
ROW=2	(2, 0) 7	(2, 1) 8	(2, 2) 9

`array1[1, :]`

02. numpy

(2023년 11월 기준)

◆ 행렬 내적(행렬 곱)



```
1 : import numpy as np
2 : array1 = np.arange(1, 5).reshape(2, 2)
3 : array2 = np.arange(5, 9).reshape(2, 2)
4 : np.dot(array1, array2)
```

```
array([[19, 22],
       [43, 50]])
```

◆ 기술통계, x 는 임의의 1차원 배열을 의미한다.

주요 메서드	설명
<code>np.mean(x)</code>	배열 x 의 평균을 구한다.
<code>np.var(x)</code>	배열 x 의 분산을 구한다.
<code>np.std(x)</code>	배열 x 의 표준편차를 구한다.
<code>np.max(x)</code>	배열 x 의 최댓값을 구한다.
<code>np.min(x)</code>	배열 x 의 최솟값을 구한다.
<code>np.median(x)</code>	배열 x 의 중간값을 구한다.
<code>np.percentile(x, y)</code>	배열 x 의 사분위수를 계산한다.

◆ 사분위 범위

주요 메서드	설명
np.mean(x)	배열 x의 평균을 구한다.
np.var(x)	배열 x의 분산을 구한다.
np.std(x)	배열 x의 표준편차를 구한다.
np.max(x)	배열 x의 최댓값을 구한다.
np.min(x)	배열 x의 최솟값을 구한다.
np.median(x)	배열 x의 중간값을 구한다.
np.percentile(x, y)	배열 x의 사분위수를 계산한다.

◆ 사분위수

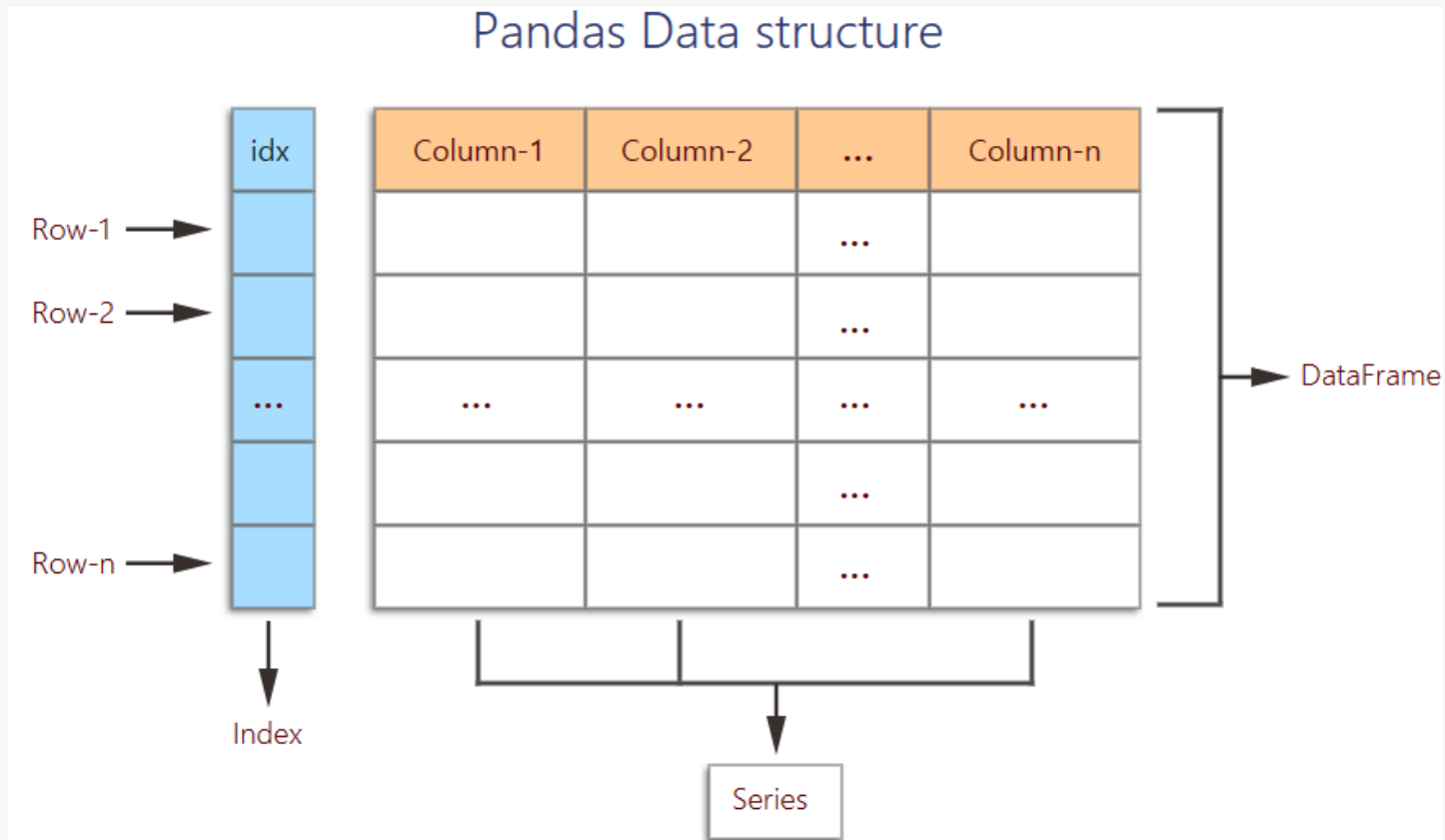
11	22	30	40	50	60	77	88	99	100	110
최솟값		25% (1사분위)		50% (2사분위)		75% (3사분위)		최댓값		

03. pandas

(2023년 11월 기준)

◆ Series vs DataFrame

- ✓ Series : 1차원 배열의 형태를 가진 자료 구조, index와 value 확인 가능
- ✓ DataFrame : 2차원 행렬 구조의 테이블 형태로 구성, index와 복수의 컬럼(Column)이 존재함



출처 : https://www.w3resource.com/w3r_images/pandas-data-structure.svg

◆ DataFrame 컬럼의 데이터 확인 및 변경

구분	내용
int	정수형, 소수점을 가지지 않은 숫자
float	실수형, 소수점 이하의 값을 가진 숫자
bool	부울형, True 혹은 False로 이루어짐
datetime	날짜와 시간 표현
category	카테고리, 범주형 변수일 경우 사용
object	문자열 & 복합형, 위의 형식으로 정할 수 없거나 여러 형식이 섞여 있는 경우 사용

◆ Series vs DataFrame

- ✓ Series : 1차원 배열의 형태를 가진 자료 구조, index와 value 확인 가능
- ✓ DataFrame : 2차원 행렬 구조의 테이블 형태로 구성, index와 복수의 컬럼(Column)이 존재함

◆ 주요 메서드 - 데이터 살펴보기, data는 pandas DataFrame을 의미한다.

주요 메서드	설명
data.head(n)	데이터프레임의 첫번째 행부터 순차적으로 n개까지의 행을 반환
data.tail(n)	데이터프레임의 마지막 행부터 역순으로 n개까지의 행을 반환
data.shape	데이터프레임의 행과 컬럼 정보를 튜플 형태로 반환
data.info()	데이터프레임의 컬럼, Non-Null 데이터 개수, 컬럼의 타입
data.describe()	컬럼별 숫자형 데이터의 개수, 평균값, 표준편차, 최솟값, 사분위수값, 최댓값을 표현함
value_counts()	해당 컬럼 값의 유형과 건수를 확인할 수 있음. 데이터의 분포도를 확인하는 데 유용함

◆ Series vs DataFrame

- ✓ Series : 1차원 배열의 형태를 가진 자료 구조, index와 value 확인 가능
- ✓ DataFrame : 2차원 행렬 구조의 테이블 형태로 구성, index와 복수의 컬럼(Column)이 존재함

◆ 주요 메서드 - 데이터 살펴보기, data는 pandas DataFrame을 의미한다.

◆ 주요 작업

- ✓ 컬럼 생성과 수정
- ✓ 데이터프레임 데이터 삭제

```
1 : data.drop("column1", axis=1) # column1 삭제
2 : data.drop(["column1", "column2"], axis=1) # column1, column2 삭제
3 : data.drop([0, 1, 2], axis=0) # 행 인덱스 0, 1, 2 삭제
```

◆ Series vs DataFrame

- ✓ Series : 1차원 배열의 형태를 가진 자료 구조, index와 value 확인 가능
- ✓ DataFrame : 2차원 행렬 구조의 테이블 형태로 구성, index와 복수의 컬럼(Column)이 존재함

◆ 주요 메서드 - 데이터 살펴보기, data는 pandas DataFrame을 의미한다.

◆ 주요 작업

- ✓ 컬럼 생성과 수정
- ✓ 데이터프레임 데이터 삭제
- ✓ 데이터 조회 (컬럼명, 슬라이싱, 논리형 인덱싱)

◆ iloc vs loc 차이

iloc	loc
Integer-location based (위치 기반)	Label(s) or Boolean Array based (명칭 기반)
<code>data.iloc[row_index, column_index]</code>	<code>data.loc[row_label, column_label]</code>
input 형태 - integer - List or Array [4, 3, 0] - Slicing 1:7 - Boolean Array	input 형태 - Single Label 5, 'a' - List or Array of Label, ["a", "b", "c"] - Slicing 'a' : 'c'

◆ 주요 메서드

주요 메서드	설명
<code>data.sort_values()</code>	DataFrame, Series의 정렬을 할 때 사용한다. SQL의 order by 키워드와 유사함.
<code>data.sum()</code>	DataFrame의 모든 컬럼 각각 합계가 나타남.

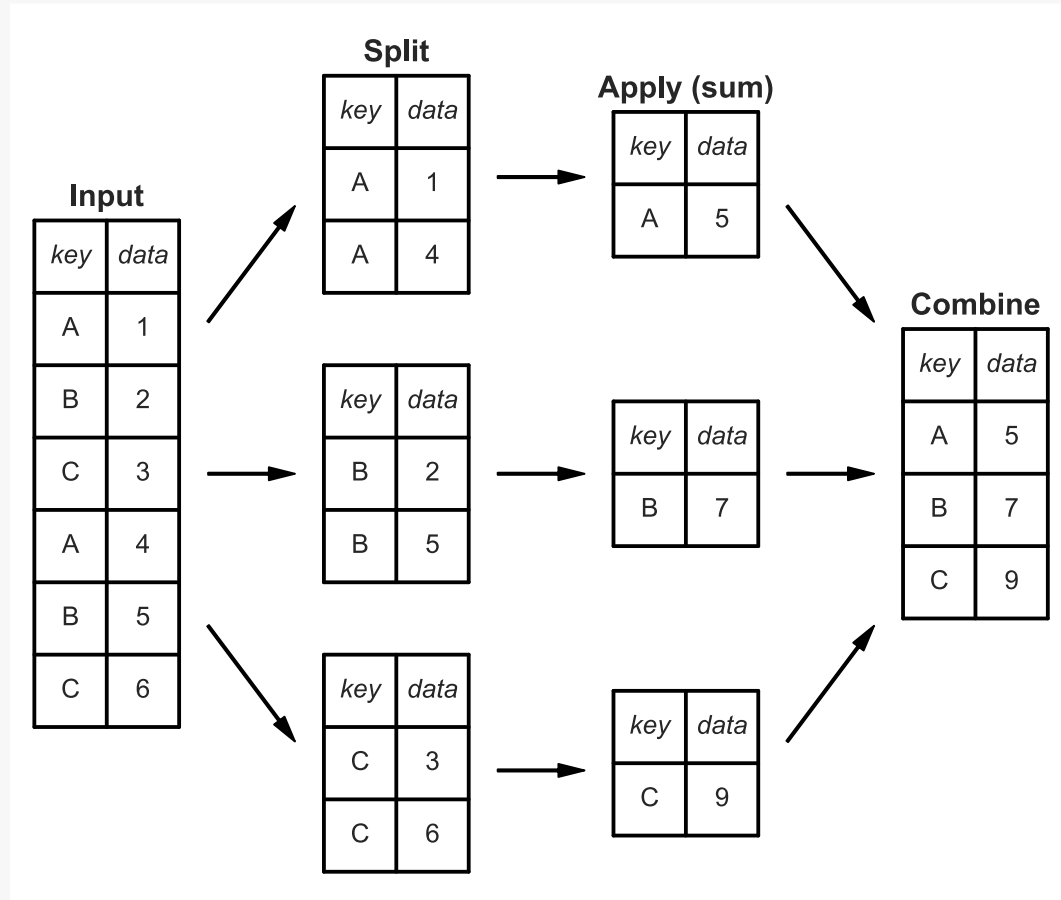
```
1 : Import seaborn as sns
2 : iris = sns.load_dataset('iris')
3 : iris[['sepal_length', 'sepal_width', 'petal_length']].sum()
```

```
sepal_length    876.5
sepal_width     458.6
petal_length     563.7
dtype: float64
```

◆ 주요 메서드

주요 메서드	설명
<code>data.sort_values()</code>	DataFrame, Series의 정렬을 할 때 사용한다. SQL의 order by 키워드와 유사함.
<code>data.sum()</code>	DataFrame의 모든 컬럼에 대하여 각각 합계가 나타남.
<code>data.min()</code>	DataFrame의 모든 컬럼에 대하여 최솟값이 나타남.
<code>data.max()</code>	DataFrame의 모든 컬럼에 대하여 최댓값이 나타남.
<code>data.count()</code>	DataFrame의 모든 컬럼에 대하여 행의 개수가 나타남
<code>data.mean()</code>	DataFrame의 모든 컬럼에 대하여 평균값이 나타남
<code>data.median()</code>	DataFrame의 모든 컬럼에 대하여 중간값이 나타남

◆ groupby 원리



◆ 집계함수 원리

주요 메서드	설명
count()	값의 개수
sum()	값들의 합
min()	최솟값
max()	최댓값
mean()	평균
median()	중앙값
std()	표준편차
var()	분산
quantile()	분위수
first()	첫번째 값
last()	마지막 값

<https://jakevdp.github.io/blog/2017/03/22/group-by-from-scratch/>

03. pandas

(2023년 11월 기준)

◆ groupby 원리

```
1 : tips.groupby(  
2 :     ['sex', 'smoker']  
3 :     ).agg(  
4 :     mean_bill = ('total_bill', 'mean'),  
5 :     median_tip = ('tip', 'median')  
6 :     ).reset_index()
```

	sex	smoker	mean_bill	median_tip
0	Male	Yes	22.284500	3.00
1	Male	No	19.791237	2.74
2	Female	Yes	17.977879	2.88
3	Female	No	18.105185	2.68

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

◆ 데이터프레임 결측치 처리

- ✓ 컬럼에 값이 없는 Null을 의미한다.
- ✓ 결측치를 처리하지 않으면 작동하지 않으므로 이 값을 반드시 다른 값으로 대체해야 함

주요 메서드	설명
<code>data.isna()</code>	결측치 여부를 확인하는 함수, 반환값은 True/False이다.
<code>data.fillna()</code>	결측치를 채우는 함수이다.

◆ 데이터 재구조화

✓ pivot_table : 많은 양의 데이터에서 필요한 자료만을 뽑아 데이터를 재구조화 할 수 있음

```
df.pivot_table(index=["ID"], columns=["반"], values = "성적", aggfunc="sum")
```

- index 피벗테이블에서 인덱스로 지정할 컬럼의 이름(두 개 이상이면 리스트로 입력할 것)
- columns 피벗테이블에서 컬럼으로 지정할 컬럼의 이름(범주형 변수 활용)
- values 피벗테이블에서 columns의 값이 될 컬럼의 이름(수치형 변수 활용)
- aggfunc 집계함수를 사용할 경우 지정

	ID	반	성적
0	1	A	100
1	1	B	88
2	1	A	85
3	2	B	75
4	2	A	100
5	2	B	80



	반	A	B
ID			
1	185	88	
2	100	155	

◆ 데이터 재구조화

✓ melt : 피벗테이블의 반대 개념으로 생각하면 된다

```
df.melt(id_vars = ["ID"], var_name = "반", value_name = "성적")
```

- id_vars 피벗 테이블에서 인덱스가 될 컬럼의 이름
- var_name variable 변수의 이름으로 지정할 문자열(선택)
- value_name value 변수의 이름으로 지정할 문자열(선택)



반	A	B
ID		
1	185	88
2	100	155

	ID	반	성적
0	1	A	92.5
1	2	A	100.0
2	1	B	88.0
3	2	B	77.5

◆ 주의

✓ 기존 테이블에서 집계된 값을 원래값으로 재분리하는 것은 아님

◆ 최댓값의 index 찾기

✓ `idxmax()` : Series or DataFrame에서 최대값이 처음 발생한 인덱스를 찾는 데 사용됨

`df.idxmax(axis=0, skipna=True, numeric_only=False)`

- `axis` 행 단위의 경우 0 또는 '인덱스', 열 단위의 경우 1 또는 '열' 을 의미
- `skipna` NA/null 값을 제외하며, 전체 행/열이 NA인 경우 결과는 NA가 됨.
- `numeric_only` Boolean, Int, Float만 포함할 것인지

시나리오	설명
Series 객체	최댓값의 index 1개만 조회 (int형으로 반환)
DataFrame 객체	다수 컬럼의 최댓값 index를 조회 (Series 객체로 반환)

◆ 날짜 데이터 처리

- ✓ 날짜와 시간을 다루기 위해서는 datetime 라이브러리 활용
- ✓ 날짜 형식으로 변환방법 (pandas DataFrame)

```
df['date'] = pd.to_datetime(df['date'], format="%Y-%m-%d %H:%M:%S")
```

◆ datetime 객체 사용 예시

```
df['date'].dt.year
```

구분	내용
year	객체 datetime의 연도를 추출함
month	객체 datetime의 월을 추출함
day	객체 datetime의 일을 추출함
hour	객체 datetime의 시간을 추출함
weekday	객체 datetime 날짜의 요일을 추출함 (Monday = 0, Sunday=6)

그 외 : <https://pandas.pydata.org/docs/reference/series.html#datetimelike-properties>

◆ Timedelta

- ✓ 두 날짜 또는 시간 간의 차이, 즉 기간을 표현함
- ✓ 두개의 datetime 컬럼(i.e., 출발시간 - 도착시간) 연산 시, **Timedelta 객체**로 자동 변환

```
df['이동시간'] = df['도착시간'] - df['출발시간']
```

```
import pandas as pd

df = pd.DataFrame({
    '출발시간' : [pd.to_datetime('2023-01-01 08:00:00')],
    '도착시간' : [pd.to_datetime('2023-01-01 09:30:00')],
})
```

```
df['이동시간'] = df['도착시간'] - df['출발시간']
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1 entries, 0 to 0
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype  
---  -
0   출발시간   1 non-null      datetime64[ns]
1   도착시간   1 non-null      datetime64[ns]
2   이동시간   1 non-null      timedelta64[ns]
dtypes: datetime64[ns](2), timedelta64[ns](1)
memory usage: 152.0 bytes
None
```

```
df.head(1)
```

	출발시간	도착시간	이동시간
0	2023-01-01 08:00:00	2023-01-01 09:30:00	0 days 01:30:00

◆ Timedelta

- ✓ 두 날짜 또는 시간 간의 차이, 즉 기간을 표현함
- ✓ 두개의 datetime 컬럼(i.e., 출발시간 - 도착시간) 연산 시, **Timedelta 객체**로 자동 변환

```
df.head(1)
```

	출발시간	도착시간	이동시간
0	2023-01-01 08:00:00	2023-01-01 09:30:00	0 days 01:30:00

df['이동시간'].dt.days

구분	내용
days	0
total_seconds()	5400(초) = hours * 3600 + minutes * 60 + seconds
dt.total_seconds() / 60	90(분) = 5400 / 60
dt.total_seconds() / (60 * 60)	1.5(시간) = 5400 / (60 * 60)

그 외 : <https://pandas.pydata.org/docs/reference/api/pandas.Timedelta.html>

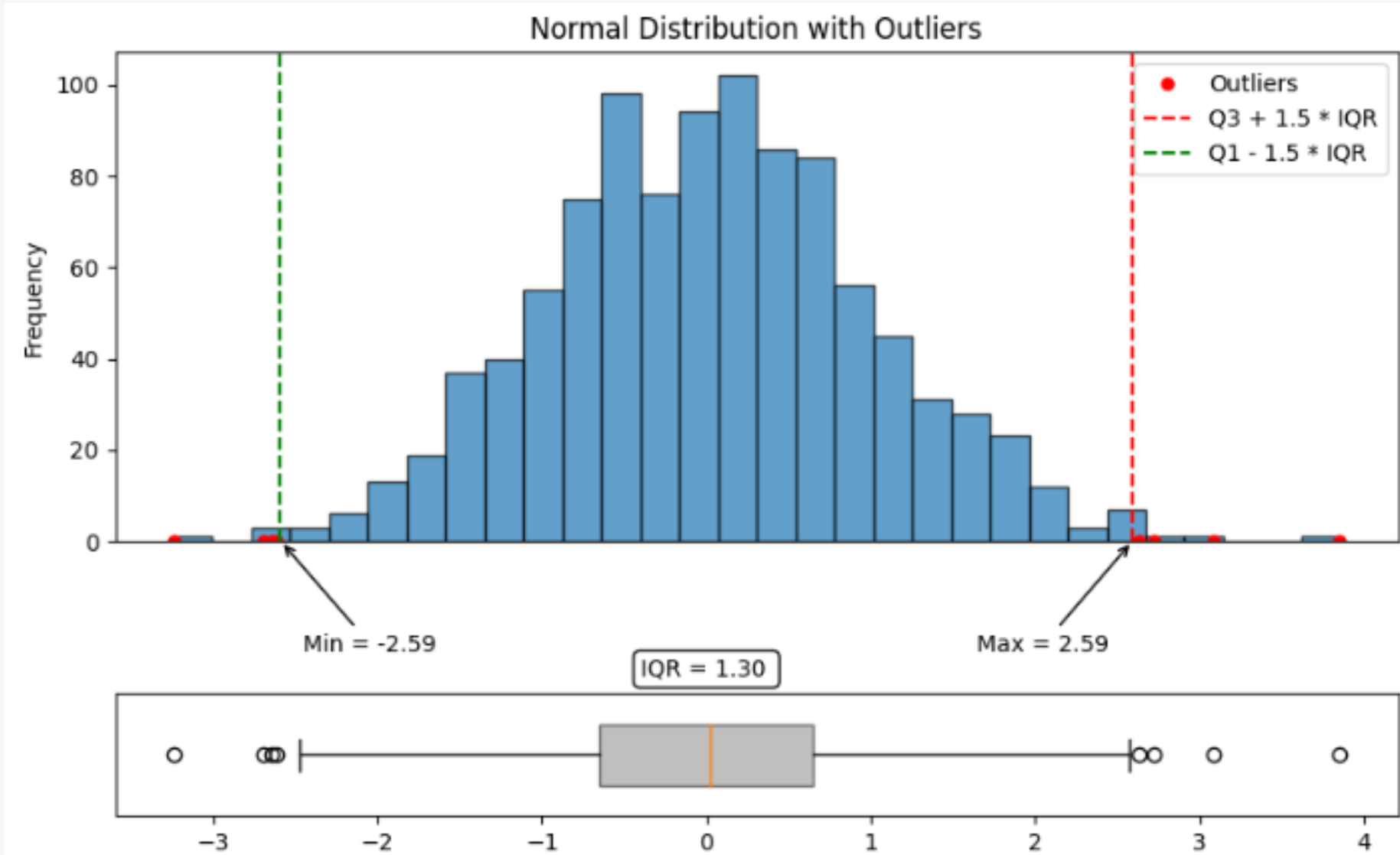
- ◆ 데이터 전처리의 의미
 - ✓ 분석에 적합하게 데이터를 가공하는 작업 의미

구분	내용
데이터 클리닝	결측치 처리, 이상치 확인 및 정제
데이터 통합	다양한 데이터 파일의 결합
데이터 변환	스케일링, 요약
데이터 축소	변수 축소, 라벨링
불균형 데이터 처리	언더 샘플링, 오버 샘플링
데이터 분할	train, test 데이터 분할

- ◆ 이상치의 의미
 - ✓ 값이 크게 차이가 나는 경우
- ◆ 이상치 발생하는 주요 원인
 - ✓ 결측치로 표시할 때 (예: error, 9999)
 - ✓ 자료 수집의 오류 시, (실내 온도 측정 예: 40000)
 - ✓ 실제 관측치, (월급 예: 100만원, 1억원)
- ◆ 이상치 처리
 - ✓ 실무에서 표준화된 정답은 없음
 - ✓ 통계적인 방법 : IQR(Inter Quantile Range) 방식

04. scikit-learn

(2023년 11월 기준)



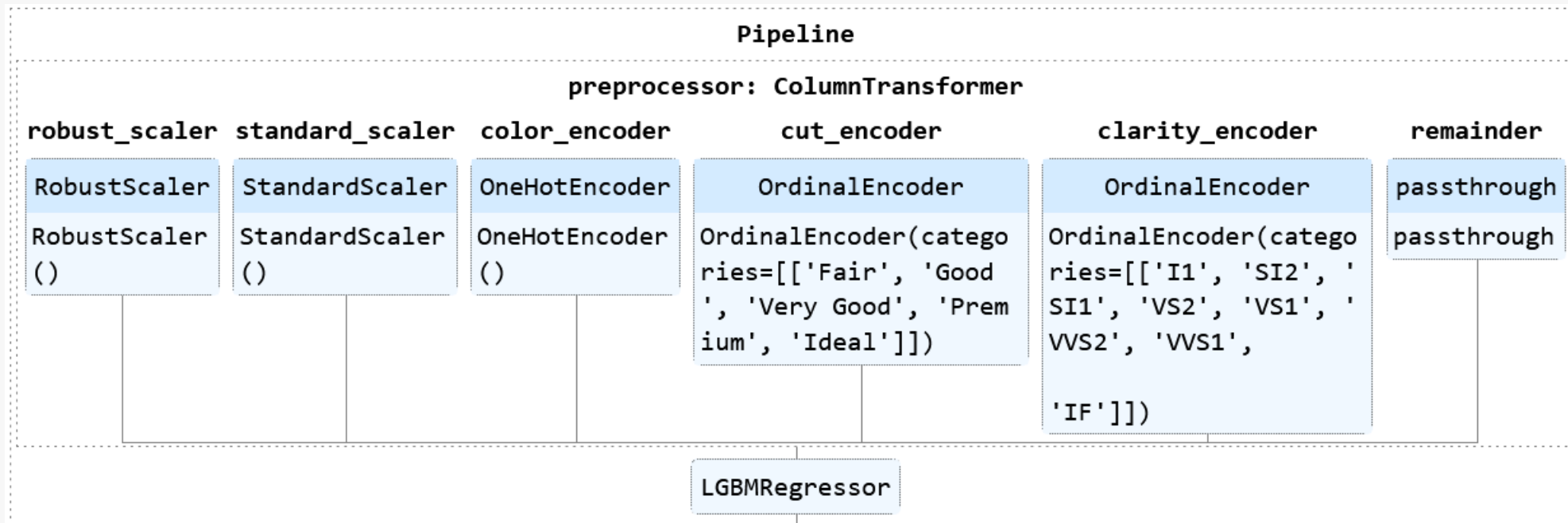
◆ 개요

✓ 머신러닝 분석 수행할 때 가장 유용하게 사용할 수 있는 파이썬 라이브러리

주요 업무	주요 모듈	설명
데이터 분리	sklearn.model_selection	• 훈련, 검증, 테스트 데이터로 데이터를 분리하기 위해 활용 • train_test_split 메서드 주로 활용
데이터 처리	sklearn.preprocessing	• Feature Scaling - StandardScaler : 평균 0, 분산 1로 기준으로 변경 - MinMaxScaler : 0과 1 사이에 위치하도록 데이터 재조정 • Binarization : 수치 데이터를 0과 1로 변환 • Encoding : 문자 데이터를 수치로 변환 • Imputer : 결측치를 채움
데이터 축소	sklearn.decomposition	• PCA 등을 통해 차원축소 지원 가능
모형 학습	sklearn.linear_model	• 선형회귀, 로지스틱 회귀 등의 알고리즘 지원
	sklearn.tree	• 트리 알고리즘 제공
	sklearn.ensemble	• 랜덤포레스트 알고리즘 제공
모형 평가	sklearn.metrics	• 분류, 회귀, 클러스터링 등에 관한 모형 평가 지표 제공

◆ 모형 자동화

- ✓ sklearn.pipeline 과 sklearn.compose 모듈을 적극 활용



◆ 머신러닝 모형 학습 전, 데이터 전처리 이유

- ✓ 머신러닝 알고리즘 적용 전, 결측치 미 허용 → 다른 고정 값 입력 필요
- ✓ 문자열 값을 허용하지 않는다. → 숫자형으로 변환
- ✓ 서로 다른 변수의 값 범위를 일정한 수준으로 변환함

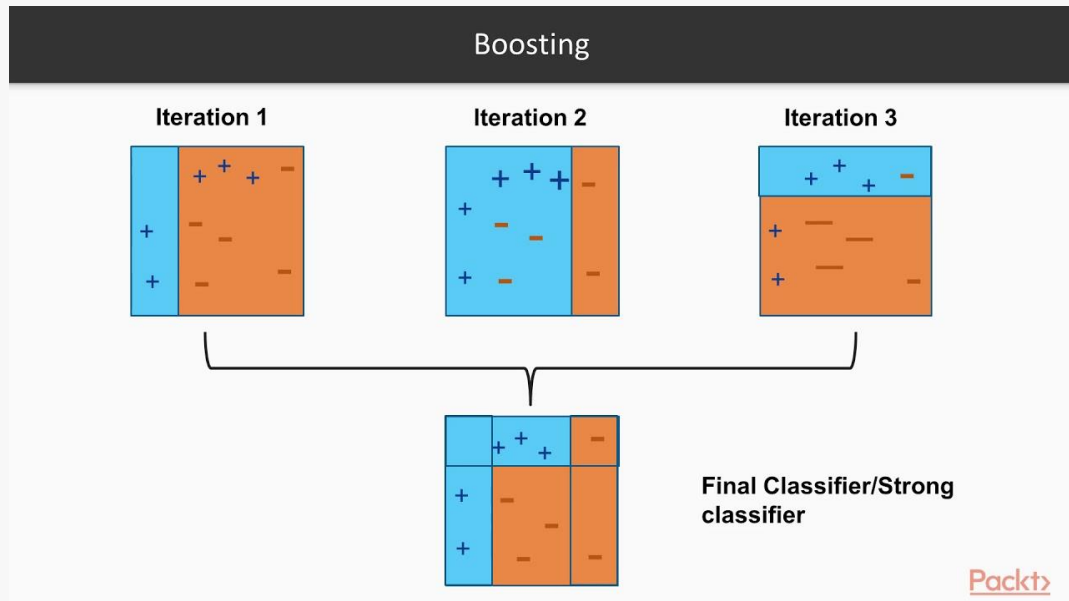
종류	모듈	설명
표준화 _{Standardization}	RobustScaler	• 중간값을 제거하고, 사분위수 범위에 따라 데이터의 배율 조정 • 1사분위 3사분위 사이의 범위 • 이상치 제거에 좋음 • quantile_range 파라미터(default [0.25, 0.75])에서 조정 가능
	StandardScaler	• 평균이 0이고 분산이 1이 정규분포를 가진 값으로 변환 • SVM, 선형회귀, 로지스틱 모델 사용 시, 필수 • 회귀보다는 분류분석에 유용
정규화 _{Normalization}	MinMaxScaler	• 모든 수치 데이터를 [0, 1], 음수가 존재하면 [-1, 1] 사이 변환 • 분류보다는 회귀에 유용
	MaxAbsScaler	• 최대절댓값과, 0이 각각 1, 0이 되도록 스케일링 하는 정규화 • 모든 값은 -1과 1사이에 표현 • 이상치에 매우 민감, • 분류보다는 회귀에 유용

◆ 머신러닝 모형 학습 전, 데이터 전처리 이유

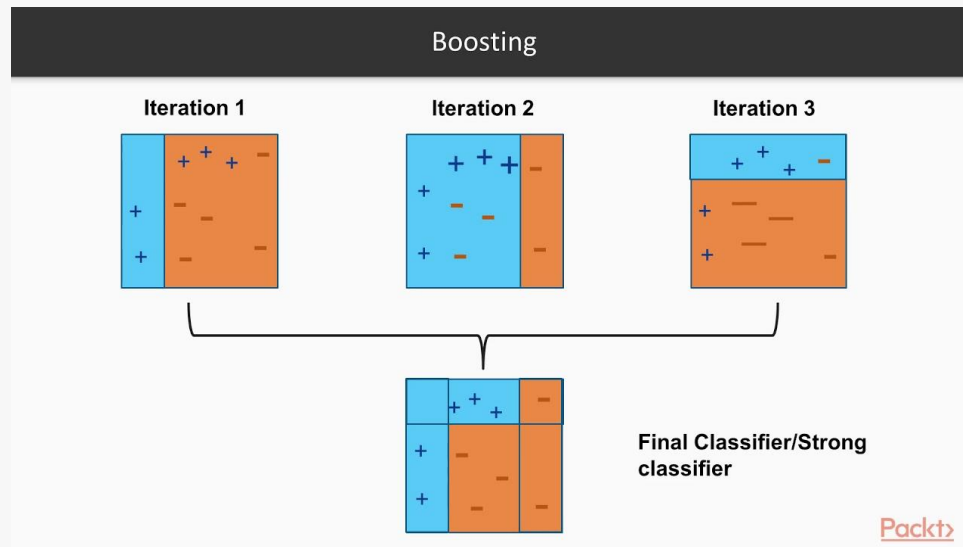
- ✓ 머신러닝 알고리즘 적용 전, 결측치 미 허용 → 다른 고정 값 입력 필요
- ✓ 문자열 값을 허용하지 않는다. → 숫자형으로 변환
- ✓ 서로 다른 변수의 값 범위를 일정한 수준으로 변환함

종류	모듈	설명
Ordinal Encoding	OrdinalEncoder	• 서열(Ordinal)척도를 숫자로 변환함 (독립변수만)
Label Encoding	LabelEncoder	• 서열(Nominal)척도를 숫자로 변환함 (종속변수만)
One-Hot Encoding	OneHotEncoder	• 명목(Nominal)척도를 숫자로 변환함 (독립변수만)

- ◆ 머신러닝 모형 : LightGBM(Light Gradient Boosting Machine)
- ◆ GBM의 기본 원리
 - ✓ 첫번째 단계의 Iteration 1을 활용하여 Y 예측
 - ✓ 발생한 잔차(residual)을 다시 Iteration 2의 input으로 넣어주고 다시 예측
 - ✓ 발생한 잔차(residual)을 다시 Iteration 3의 input으로 넣어주고 다시 예측
 - ✓ residual은 점차 계속 작아질 것임



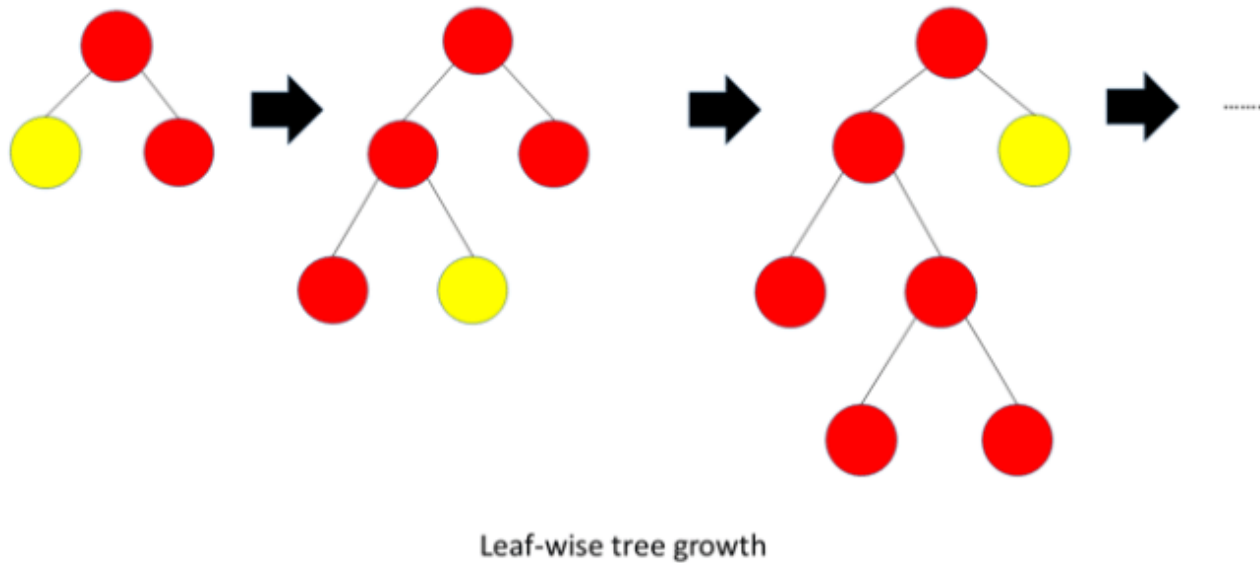
- ◆ 머신러닝 모형 : LightGBM(Light Gradient Boosting Machine)
- ◆ GBM의 기본 원리
 - ✓ 첫번째 단계의 Iteration 1을 활용하여 Y 예측
 - ✓ 발생한 잔차(residual)을 다시 Iteration 2의 input으로 넣어주고 다시 예측
 - ✓ 발생한 잔차(residual)을 다시 Iteration 3의 input으로 넣어주고 다시 예측
 - ✓ residual은 점차 계속 작아질 것임
 - ✓ 이렇게 만들어진 Iteration 1 + Iteration 2 + Iteration 3이 우리의 GBM이 됨



◆ 머신러닝 모형 : LightGBM(Light Gradient Boosting Machine)

◆ 기본원리

- ✓ 높은 성능 : LightGBM은 대용량 데이터셋에서도 빠른 학습과 예측 제공, 적은 메모리 사용
- ✓ Leaf-Wise 트리 분할 : 트리의 균형을 유지하지 않고 최대손실을 갖는 리프 노드 우선 분할하여 더 정확한 예측 가능



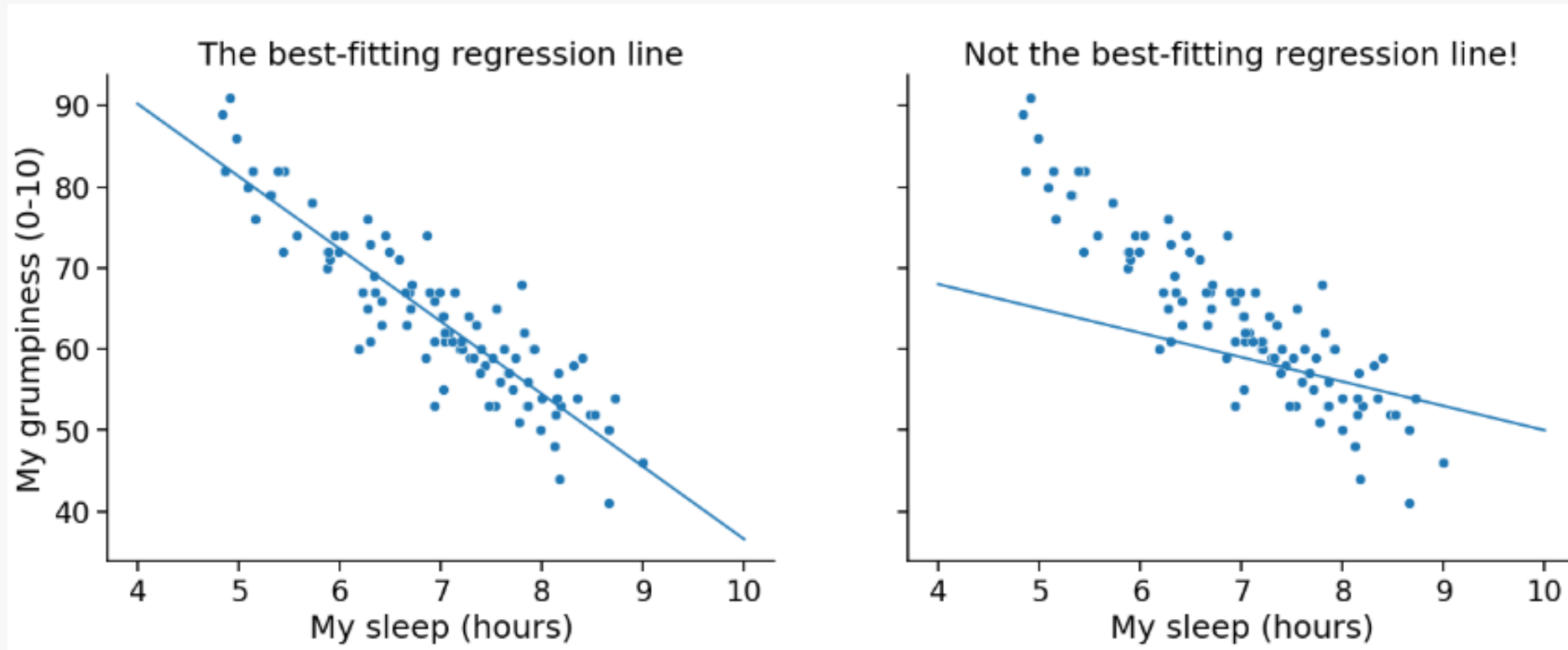
◆ 머신러닝 모형 : LightGBM(Light Gradient Boosting Machine)

◆ 하이퍼 파라미터 튜닝 주요 매개변수

주요 매개변수	설명
learning_rate	- 데이터 타입(기본값) : float(default = 0.1), - 각 부스팅의 반복 시, 분류기에 적용되는 가중치
n_estimators	- 데이터 타입(기본값) : int(default=100) - 설명 : LightGBM에서 부스팅 반복 횟수.
max_depth	- 데이터 타입(기본값) : int(default=-1) - 트리 모델의 최대 깊이를 제한. 이는 데이터가 작을 때 과적합을 처리하는 데 사용됨.
num_leaves	- 데이터 타입(기본값) : int(default=31) - 한 트리의 최대 잎 수
min_split_gain	- 데이터 타입(기본값) : double(default = 0.0) - 분할을 수행하기 위한 최소 이득 / 훈련 속도를 높이는데 사용
subsample	- 데이터 타입(기본값) : double(default = 1.0) - 리샘플링 없이 데이터의 일부를 무작위로 선택 - 모형학습 시간 및 과적합을 제어하는데 사용함
random_state	- 데이터 타입(기본값) : int(default=None) - 추정량의 임의성을 제어함

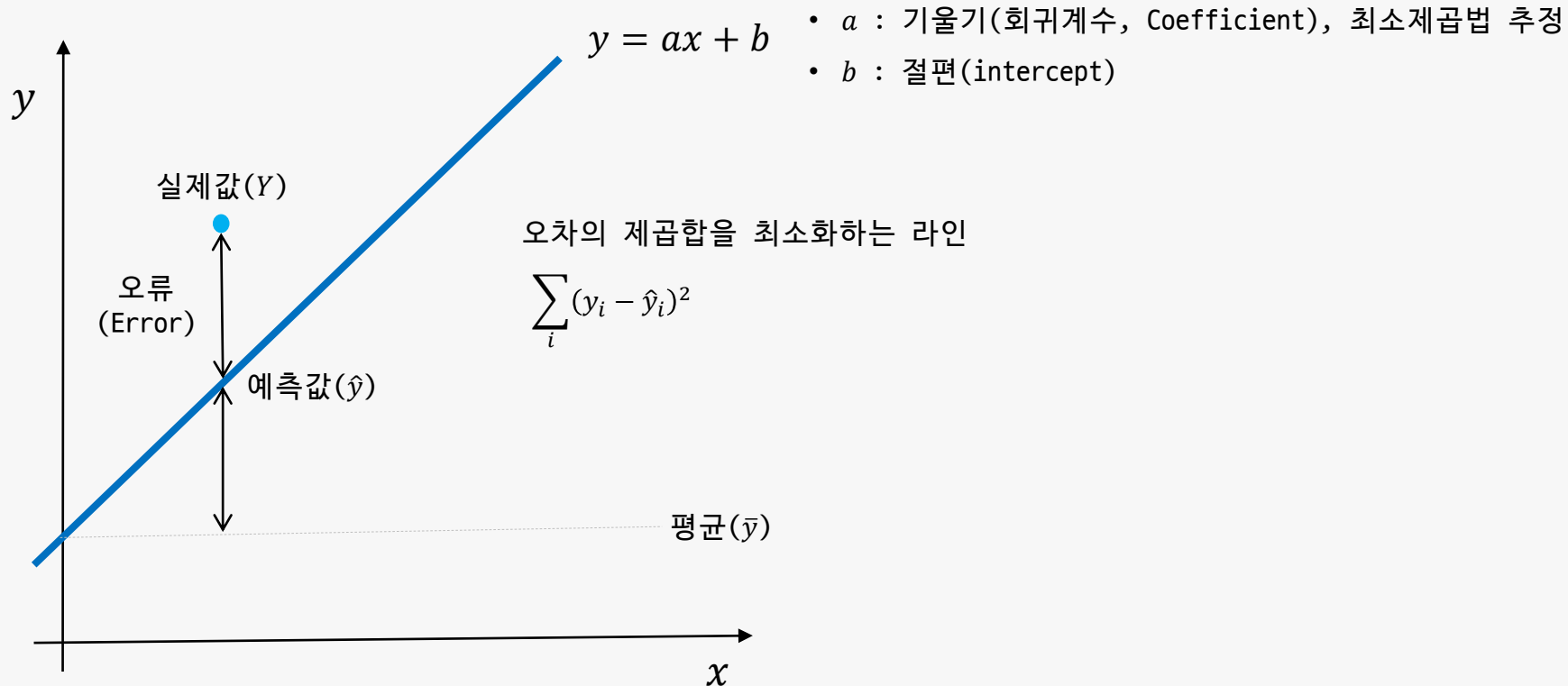
◆ 성능평가 - 회귀 모형 오차의 개념

- ✓ 오차는 실제값과 예측값의 차이를 말함, 양(+)의 값과 음(-)의 값 발생
- ✓ 양과 음의 값 합산 시, 오차는 0에 가까워짐, 따라서, 제곱 또는 절댓값을 취함
- ✓ 회귀 모형은 오차의 제곱 혹은 절댓값의 합이 최소화되는 라인을 찾는 것이 목표임



◆ 성능평가 - 회귀 모형 오차의 개념

- ✓ 오차는 실제값과 예측값의 차이를 말함, 양(+)의 값과 음(-)의 값 발생
- ✓ 양과 음의 값 합산 시, 오차는 0에 가까워짐, 따라서, 제곱 또는 절댓값을 취함
- ✓ 회귀 모형은 오차의 제곱 혹은 절댓값의 합이 최소화되는 라인을 찾는 것이 목표임



◆ 성능평가 - 회귀 : MAE(Mean Absolute Error)

✓ 실제값과 예측값의 차이 활용

정 의	• 실제값과 예측값의 차이를 절댓값으로 변환해 평균한 것
수 식	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
특 징	• 에러의 크기가 그대로 반영 • 이상치에 영향 받음

◆ Python Code

```
1 : from sklearn.metrics import mean_absolute_error
2 : mae = mean_absolute_error(y_test, y_pred)
    0.00
```

◆ 성능평가 - 회귀 : MSE(Mean Squared Error)

✓ 실제값과 예측값의 차이 활용

정 의	• 실제값과 예측값의 차이를 제공해 평균한 것
수 식	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
특 징	• 실제값과 예측값 차이의 면적 합을 의미 • 특이값이 존재하면 수치가 증가

◆ Python Code

```
1 : from sklearn.metrics import mean_squared_error
2 : mae = mean_squared_error (y_test, y_pred)
    0.00
```

◆ 성능평가 - 회귀 : RMSE(Root Mean Squared Error)

✓ 실젯값과 예측값의 차이 활용

정 의	• 실젯값과 예측값의 차이를 제공해 평균한 것에 루트를 씌운 것
수 식	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
특 징	• 오차에 제곱을 하면 오차가 클수록 그에 따른 가중치가 높게 반영 • 이 때, 손실이 기하급수적으로 증가하는 상황에서 실제 오류값의 평균보다 값이 더 커지지 않도록 상쇄하기 위해 사용

◆ Python Code

```
1 : from sklearn.metrics import mean_squared_error
2 : import numpy as np
3 : mse = mean_squared_error(y_test, y_pred)
4 : rmse = np.sqrt(mse)
```


◆ 성능평가 - 회귀 : MSLE(Mean Squared Log Error)

✓ 실제값과 예측값의 차이 활용

정 의	• 실제값과 예측값의 차이를 제공해 평균한 것에 로그를 적용
수 식	$MSLE = \log \left(\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \right)$
특 징	• RMSE와 같이 오차가 기하급수적으로 증가하는 상황에서 실제 오차의 평균보다 값이 더 커지지 않도록 상쇄

◆ Python Code

```

1  :  from sklearn.metrics import mean_squared_log_error
2  :  msle = mean_squared_log_error(y_test, y_pred)
3  :  msle
    0.00
    
```

◆ 성능평가 - 회귀 : MAPE(Mean Absolute Error)

✓ 실제값과 예측값의 차이 활용

정 의	• MAE를 퍼센트로 반환한 것
수 식	$MAPE = \frac{n}{100} \sum_{i=1}^n \left \frac{y_i - \hat{f}(x_i)}{y_i} \right $
특 징	• 오차가 예측값에서 차지하는 정도를 나타냄

◆ Python Code

```
1 : import numpy as np
2 : def MAPE(y_test, y_pred):
3 :     mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
4 :     return mape
5 : mape = MAPE(y_test, y_pred)
6 : mape
```

- ◆ 성능평가 - 회귀 : 결정계수(R^2)
 - ✓ 실젯값과 예측값의 차이 활용

정 의	- 회귀 모형이 선형인 경우에는 결정계수를 평가 지표로 활용 - 결정계수는 0에서 1사이의 값을 가지며 1에 가까울수록 모형의 설명력이 좋음
수 식	$R^2 = \frac{SSR}{SST} = \frac{SST - SSE}{SST} = 1 - \frac{SSE}{SST}$

지표	설명	수식
SST (Total Sum of Squared)	- 전체제곱합 - 실제 관측치(y_i)와 y값들의 평균($\bar{y}$)의 차이를 제공하여 합한 값 y가 가지는 전체 변동	$\sum_{i=1}^n (y_i - \bar{y})^2$
SSR (Regression Sum of Squared)	- 회귀제곱합 - 모형 예측치($\hat{y}_i$)와 y값들의 평균($\bar{y}$)의 차이를 제공하여 합한 값 y가 가지는 전체 변동성 중 회귀 모형으로 설명할 수 있는 변동	$\sum_{i=1}^n (\hat{y}_i - \bar{y})^2$
SSE (Error Sum of Squared)	- 오차제곱합 - 실제 관측치(y_i)와 모형 예측치($\hat{y}_i$)의 차이를 제공하여 합한 값 y가 가지는 전체 변동성 중 회귀 모형으로 설명할 수 없는 변동	$\sum_{i=1}^n (y_i - \hat{y}_i)^2$

◆ 성능평가 - 회귀 : 결정계수(R^2)

✓ 실제값과 예측값의 차이 활용

정 의	- 회귀 모형이 선형인 경우에는 결정계수를 평가 지표로 활용 - 결정계수는 0에서 1사이의 값을 가지며 1에 가까울수록 모형의 설명력이 좋음
수 식	$R^2 = \frac{SSR}{SST} = \frac{SST - SSE}{SST} = 1 - \frac{SSE}{SST}$

◆ Python Code

```
1 : from sklearn.metrics import r2_score
2 : r2 = r2_score(y_test, y_pred)
3 : r2
    0.92
```

◆ 성능평가 - 분류 : 혼동행렬

		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

◆ 용어 정리

- TN(True Negative) : 음성을 음성이라고 예측
- FP(False Positive) : 음성을 양성이라고 예측
- FN(False Negative): 양성을 음성이라고 예측
- TP(True Positive) : 양성을 양성이라고 예측

◆ 성능평가 - 분류 : 혼동행렬

- ✓ 정확도의 한계점 보완하기 위해 혼동행렬 활용

정 의	• 모델의 성능을 평가할 때 사용되는 지표, 예측값과, 실제값을 보여주는 행렬
특 징	• 4분면 행렬에서 실제 레이블 클래스 값과 예측 레이블 클래스 값이 어떤 유형을 가지고 매핑되는지 나타남

◆ Python Code

```
1 : from sklearn.metrics import confusion_matrix
2 : cm = confusion_matrix(y_test, y_pred)
3 : cm
```

◆ 성능평가 - 분류 : 정확도(Accuracy)

- ✓ 실제분류와 예측분류가 얼마나 일치했는가를 기반으로 알고리즘의 성능을 평가

정 의	• 실제 데이터에서 예측 데이터가 얼마나 같은지 판단하는 지표
수 식	$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$
특 징	• 데이터 구성에 따라 머신러닝 모델의 성능을 왜곡할 가능성 있음

◆ Python Code

```
1 : from sklearn.metrics import accuracy_score
2 : acc = accuracy_score(y_test, y_pred)
3 : acc
```

		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

◆ 성능평가 - 분류 : 정밀도(Precision)

✓ Positive 데이터 예측에 집중한 평가지표

정 의	• Positive로 예측한 것들 중 실제로도 Positive인 것들의 비율
수 식	$Precision = \frac{TP}{FN + TP}$
특 징	• Positive 예측성능을 더욱 정밀하게 측정하기 위한 평가지표 • 양성 예측도라 불리움 • 정밀도가 상대적인 중요성을 가지는 경우 - 실제 Negative인 데이터를 Positive로 잘못 예측했을 때 업무상 큰 영향 발생 시

◆ Python Code

```

1  : from sklearn.metrics import precision_score
2  : acc = precision_score(y_test, y_pred)
3  : acc
    
```

		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

◆ 성능평가 - 분류 : 재현율(Recall)

✓ Positive 데이터 예측에 집중한 평가지표

정 의	• 실제 Positive인 것들 중 Positive로 예측한 것들의 비율
수 식	$Recall = \frac{TP}{FP + TP}$
특 징	- 민감도(Sensitivity) 또는 TPR(True Positive Rate)라고 불림 - 재현율이 상대적인 중요성을 가지는 경우 - 실제 Positive인 데이터를 Negative로 잘못 예측 했을 때 업무상 큰 영향이 발생할 때

◆ Python Code

```

1 : from sklearn.metrics import recall_score
2 : recall = recall_score(y_test, y_pred)
3 : recall
    
```

		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

- ◆ 성능평가 - 분류 : F1 스코어
 - ✓ 정밀도와 재현율을 결합한 성능지표

정 의	• 실제 Positive인 것들 중 Positive로 예측한 것들의 비율
수 식	$F1\ score = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 \times \frac{precision \times recall}{precision + recall}$
특 징	• 정밀도와 재현율이 어느 한쪽으로 치우치지 않고 적절한 조화를 이룰 때 상대적으로 높은 수치를 나타냄

◆ Python Code

```
1 : from sklearn.metrics import f1_score
2 : f1 = f1_score(y_test, y_pred)
3 : f1
```

		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

- ◆ 성능평가 - 분류 : ROC-Curve
 - ✓ ROC 곡선과 이를 기반으로 하는 AUC 스코어는 이진 분류모델의 주요 성능평가지표임

정 의	FPR(False Positive Rate)이 변할 때, TPR(True Positive Rate)이 변하는 것을 나타내는 곡선(ROC)
수 식	$TNR = \frac{TN}{FP + TN}$$FPR = 1 - TNR = \frac{FP}{FP + TN}$
특 징	- TPR을 y축으로, FPR을 x축으로 하는 그래프 - 분류 결정 임계값을 조절하면서 FPR이 0부터 1까지 변할 때, TPR의 변화값을 그래프에 나타냄 - 우상향 그래프로 그려짐

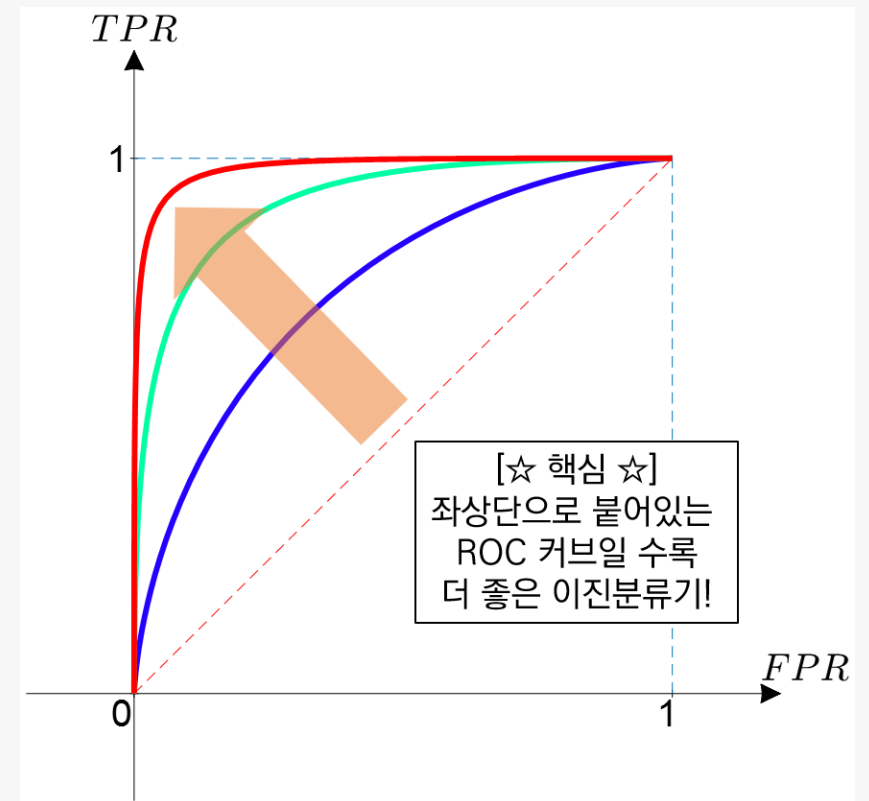
		예측 결과	
		TRUE	FALSE
실제 정답	TRUE	TP (True Positive)	FN (False Negative)
	FALSE	FP (False Positive)	TN (True Negative)

◆ 성능평가 - 분류 : ROC-Curve

✓ ROC 곡선과 이를 기반으로 하는 AUC 스코어는 이진 분류모델의 주요 성능평가지표임

◆ Python Code

```
1 : from sklearn.metrics import roc_curve
2 : import matplotlib.pyplot as plt
3 : fpr, tpr, thres = roc_curve(y_test, y_pred, pos_label = 1)
4 : plt.plot(fpr, tpr)
5 : plt.show()
```



◆ 성능평가 - 분류 : AUC 스코어

- ✓ ROC 곡선 아래의 면적 값을 분류 성능지표로 사용

정 의	• Area Under the ROC Curve • ROC 곡선 아래의 면적 • 1에 가까울수록 예측성능이 우수하다고 판단
특 징	• AUC 값이 커지려면 FPR이 작을 때 TPR 값이 커야 함 • 우상향 직선에서 멀어지고 왼쪽 상단의 모서리 쪽으로 가파르게 곡선이 이동할수록 AUC가 1에 가까워짐 • 랜덤 수준의 AUC값은 0.5

◆ Python Code

```
1 : from sklearn.metrics import roc_curve, auc
2 : fpr, tpr, thres = roc_curve(y_test, y_pred, pos_label = 1)
3 : auc = auc(fpr, tpr)
5 : auc
```

작업형 제3유형

with  python™

◆ 가설 (Hypothesis)

✓ 모집단의 특성, 특히 모수에 대한 가정 혹은 잠정적인 결론이다.

종류	설명
귀무가설 (H_0 , Null Hypothesis)	- 기존과 비교하여 변화 혹은 차이가 없음을 나타내는 가설 - 검정 방법에 따라 귀무가설의 내용이 달라짐
대립가설 (H_1 , Alternative Hypothesis)	- 표본을 통해 확실한 근거를 가지고 입증하고자 하는 가설 - 연구가설 (Research Hypothesis) 이라고도 함

◆ 가설 검정 시, 관심 있는 가설은 대립가설이며, 대립가설이 참이라는 확실한 근거가 없다면 귀무가설 채택

- ✓ 표현문구는 귀무가설 채택 (X), 귀무가설을 기각하지 못한다 (O)
- ✓ 대립가설이 참이라는 확실한 근거 발견 시, 귀무가설을 기각한다(O) 표현

01. 가설 검정 주요 용어

(2023년 11월 기준)

◆ 가설 검정

- ✓ 단계 1. 모집단에 대해 어떤 가설 설정
- ✓ 단계 2. 모집단으로부터 추출된 표본 분석
- ✓ 단계 3. 연구 가설이 틀린지 맞는지 타당성 여부를 검정하는 통계적 기법

◆ 검정통계량

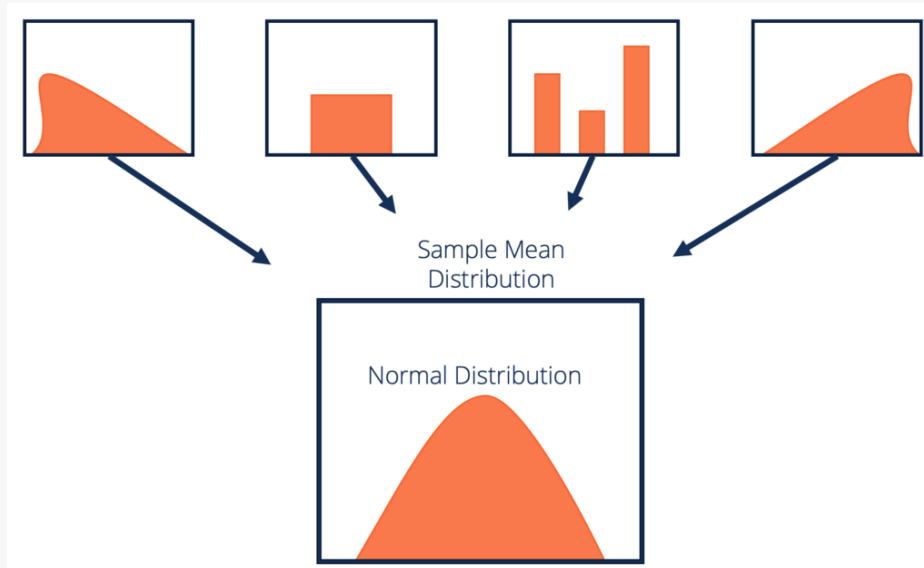
- ✓ 모수 추론하기 위해 필요한 표본 통계량
- ✓ 귀무가설이 참이라는 전제하에 추출된 확률표본의 정보를 이용하여 계산함
- ✓ 중심극한정리 이론에 근거하기 때문에, 적은 표본으로도 전체 모수를 추정할 수 있음

01. 가설 검정 주요 용어

(2023년 11월 기준)

◆ 중심 극한 정리(CLT : Central Limit Theorem)

- ✓ 정의 : 정규분포가 아닌 분포라도 선택된 표본들의 평균을 모아 다시 분포를 그리면 정규분포가 됨
- ✓ 단, 최소 표본의 크기가 작으면 안됨 ($n \geq 30$)
- ✓ CLT에 따르면, Sampling Distribution은 정규분포를 따름
- ✓ 모든 모집단 데이터의 평균 = 모든 샘플 평균들의 평균 = Sampling Distribution의 평균



출처 : <https://corporatefinanceinstitute.com/resources/data-science/central-limit-theorem/>

01. 가설 검정 주요 용어

(2023년 11월 기준)

◆ 가설 검정 오류

✓ 다음과 같은 통계적 오류가 발생할 가능성이 항상 존재함

	귀무가설(H_0)이 사실이라고 판정 (무죄)	귀무가설(H_0)이 거짓이라고 판정 (유죄)
귀무가설(H_0)이 사실	옳은 결정 No Error ($1-\alpha$) 무죄를 무죄라 함	제1종 오류 무죄를 유죄라 함
귀무가설(H_0)이 거짓	제2종 오류 유죄를 무죄라 함	옳은 결정 No Error ($1-\beta$) 유죄를 유죄라 함

01. 가설 검정 주요 용어

(2023년 11월 기준)

◆ 가설 검정 오류

✓ 다음과 같은 통계적 오류가 발생할 가능성이 항상 존재함

종류	설명
제 1종 오류 (Type I Error)	- 귀무가설이 참이나 귀무가설을 기각하는 오류 - 유의수준(Level of Significance) - 제 1종 오류를 범할 최대 허용확률 α로 표기하며, 일반적으로 α값을 0.01, 0.05 또는 0.1 등의 값으로 설정 - 신뢰수준(Level of Confidence) - 귀무가설이 참일 때 이를 참이라고 판단하는 확률 $1 - \alpha$로 표기함
제 2종 오류 (Type II Error)	- 귀무가설이 참이 아닌데, 귀무가설을 채택하는 경우 - 베타수준(β) - 제2종 오류를 범할 최대 허용확률 - 검정력 - 귀무가설이 참이 아닌 경우 이를 기각할 수 있는 확률 $1 - \beta$로 표기함

- ◆ 의사 결정의 원칙
 - ✓ 귀무가설이 옳다는 가정하에 H_1 이 옳다는 증거를 제시해야 함 (Presumed Innocence)
- ◆ 의사 결정의 기준
 - ✓ 가장 주의해야 하는 것은 1종 오류를 범하는 것
 - 나의 주장이 옳지 않은데 옳다고 결론 내리는 것
 - ✓ 의사결정의 기준은 1종 오류를 범할 확률(유의 수준 α)

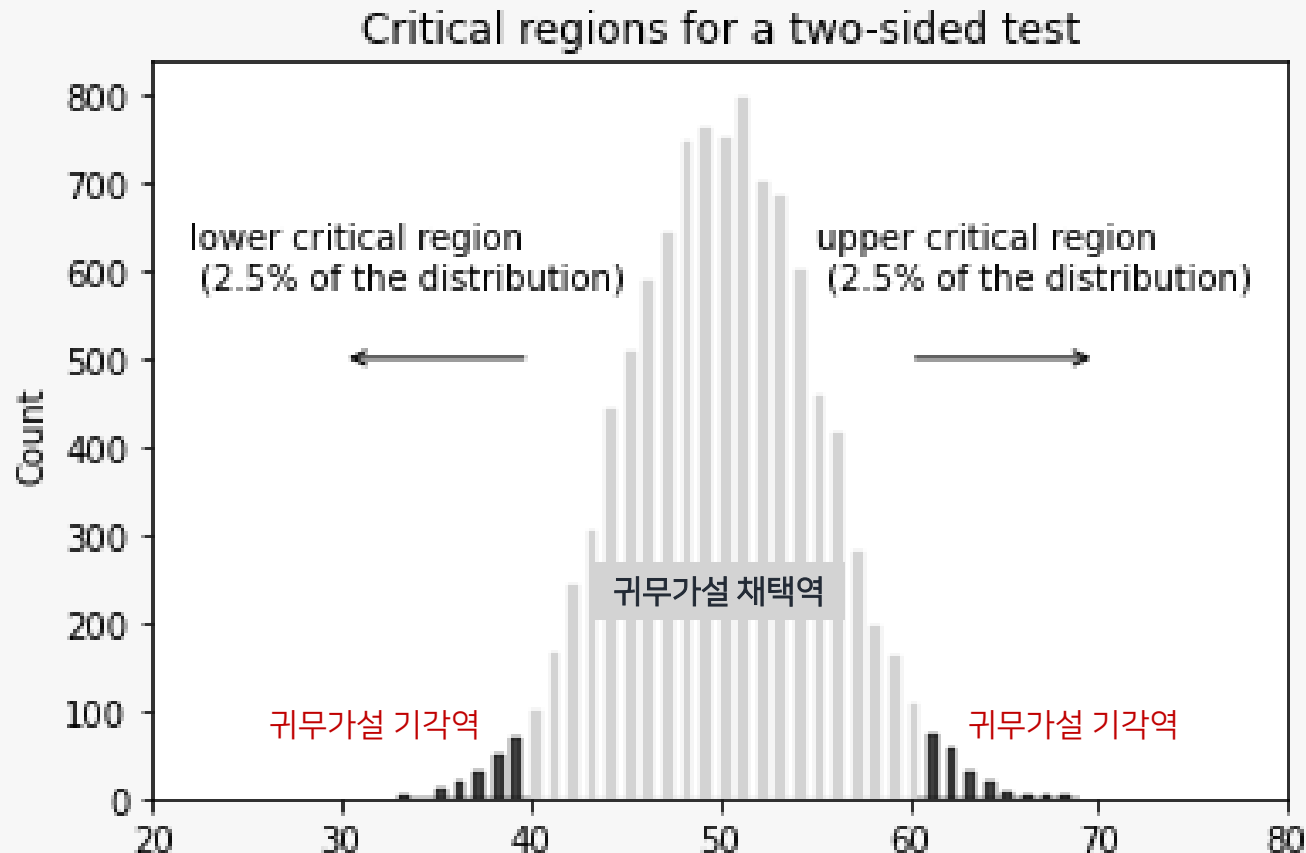
	의사결정	설명
$\alpha = 0$	H_0	무죄인 사람을 유죄라고 판단 할 오류 0%
$\alpha = 0.01$ (1%)	H_0 : 99% H_1 : 1%	매우 보수적이며 엄격한 판단 표본에서 구한 통계량 값이 나올 확률이 1%보다 작을 때 의미
$\alpha = 0.05$ (5%)	H_0 : 95% H_1 : 5%	오류가 5%까지 허용 상대적으로 덜 엄격한 의사결정

01. 가설 검정 주요 용어

(2023년 11월 기준)

◆ p-value와 임계값

- ✓ 귀무가설 H_0 가 참일 때 표본에서 얻어진 결과가 귀무가설을 기각하게 하는 확률
- ✓ 주어진 유의수준 α 하에서 귀무가설 H_0 의 채택 또는 기각 여부를 판정하여 주는 기준이 되는 값



◆ p-value와 임계값

- ✓ 귀무가설 H_0 가 참일 때 표본에서 얻어진 결과가 귀무가설을 기각하게 하는 확률
- ✓ 주어진 유의수준 α 하에서 귀무가설 H_0 의 채택 또는 기각 여부를 판정하여 주는 기준이 되는 값

p-value	Sig. stars	해석	귀무가설
$p > 0.05$		통계적으로 유의하지 않음	채택(Retained)
$p < 0.05$	*	통계적으로 유의함($\alpha = 0.05$)	기각(Rejected)
$p < 0.01$	**	통계적으로 유의함($\alpha = 0.01$)	기각(Rejected)
$p < 0.001$	***	통계적으로 유의함($\alpha = 0.001$)	기각(Rejected)

02. 가설 검정 기법 - 단일표본 T-검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 모평균 μ 에 대한 검정 절차로 다음과 같이 귀무가설과 대립가설 설정

가설	표현식	설명
H_0	$\mu = \mu_0$	모평균과 표본평균은 같다
H_1	$\mu \neq \mu_0$	모평균과 표본평균은 같지 않다.

◆ 검정통계량

- ✓ 모분산을 알고 있을 때,

$$Z = \frac{\bar{X} - \mu}{\sigma / \sqrt{n}}$$

$\bar{X}$: 표본평균

σ : 모분산

n : 표본의 갯수

02. 가설 검정 기법 - 단일표본 T-검정

(2023년 11월 기준)

◆ 주요 메서드 - 실습

- ✓ 모평균의 값을 알고 있을 때

```
1 : from scipy import stats
2 : t_score, p_value = stats.ttest_1samp(df['Height'], popmean=75)
3 : print(round(t_score, 2), round(p_value, 2))
    0.87, 0.39
```

◆ 메서드 설명

- ✓ 독립 관측치 샘플 array의 예상값(평균)이 주어진 모집단 평균인 popmean과 같다는 귀무 가설에 대한 검정.

메서드	매개변수	설명
<code>ttest_1samp(array, popmean)</code>	array	Sample Observation
	popmean	Expected value in null hypothesis

03. 가설 검정 기법 - 독립표본 T-검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 두개의 독립 표본 X, Y 가 각각 $N(\mu_1, \sigma_1^2), N(\mu_2, \sigma_2^2)$ 를 따를 때, 두 모집단의 평균 차이 $\mu_1 - \mu_2$ 의 검정은 다음과 같음.

가설	표현식	설명
H_0	$\mu_1 = \mu_2$	그룹 1의 평균과 그룹 2의 평균은 같다.
H_1	$\mu_1 \neq \mu_2$	그룹 1의 평균과 그룹 2의 평균은 같지 않다.

◆ 가정(Assumptions) 확인

- ✓ 독립성
- ✓ Normality 정규성
- ✓ Homogeneity of Variance 분산의 동질성

03. 가설 검정 기법 - 독립표본 T-검정

(2023년 11월 기준)

◆ 정규성 검정

- ✓ 각 그룹의 표본수가 $N \leq 30$ 이하일 때, 검정해야 함
- ✓ 각 그룹의 표본수가 모두 $N \geq 30$ 이상일 때, 중심극한정리에 의해 정규성 가정을 만족했다고 봄

가설	설명
H_0	각 자료는 정규분포를 따른다.
H_1	각 자료는 정규분포를 따르지 않는다.

◆ 확인 방법

- ✓ Shapiro-Wilk tests

◆ 가정 위반 시, Mann-Whitney Test를 진행

03. 가설 검정 기법 - 독립표본 T-검정

(2023년 11월 기준)

◆ 등분산성 검정

- ✓ 각 그룹의 데이터수가 다르면 분산이 같은지 검정
- ✓ 만약에 각 그룹의 데이터수가 동일하면, 분산이 같다고 가정함

가설	설명
H_0	두 그룹의 분산은 차이가 없음
H_1	두 그룹의 분산은 차이가 있음

◆ 확인 방법

- ✓ bartlett, fligner, levene 검정 주로 사용
- ✓ 일반적으로는 levene 검정을 주로 사용

◆ 가정 위반 시

- ✓ Welch Test 사용

◆ 가설 설정

✓ 두개의 독립 표본 X, Y 가 각각 $N(\mu_1, \sigma_1^2), N(\mu_2, \sigma_2^2)$ 를 따를 때, 두 모집단의 평균 차이 $\mu_1 - \mu_2$ 의 검정은 다음과 같음.

가설	표현식	설명
H_0	$\mu_1 = \mu_2$	그룹 1의 평균과 그룹 2의 평균은 같다.
H_1	$\mu_1 \neq \mu_2$	그룹 1의 평균과 그룹 2의 평균은 같지 않다.

◆ 검정통계량

- ✓ 그룹 1의 표본평균 $\bar{X}$, 그룹 2의 표본평균 $\bar{Y}$ 의 차이에 근거하여 구성
- ✓ 검정통계량 T 는 자유도 $n + m - 2$ 인 t -분포를 따른다.

$$T = \frac{\bar{X} - \bar{Y}}{S_p \sqrt{\frac{1}{n} + \frac{1}{m}}}, S_p^2 = \frac{(n-1)s_1^2 + (m-1)s_2^2}{n+m-2}$$

S_p^2 : 공통분산 σ^2 의 합동 표본분산

$n - 1$: 그룹 1 표본의 자유도

$m - 1$: 그룹 2 표본의 자유도

◆ 주요 메서드 - 등분산성 검정 실습

- ✓ 데이터의 각 변수들은 정규분포를 만족하며, 두 그룹은 등분산을 만족한가?

```
1 : from scipy import stats
2 : stats.levene(df.loc[df['supp'] == "VC", 'len'], df.loc[df['supp'] == "OJ", 'len'])
LeveneResult(statistic=1.2135720656945064, pvalue=0.2751764616144053)
```

◆ 메서드 설명

- ✓ Levene 검정은 모든 입력 샘플이 동일한 분산을 가진 모집단에서 나온 것이라는 귀무가설을 테스트 함

메서드	매개변수	설명
<code>levene(sample1, sample2, ...)</code>	sample1 : array_like	The sample data, possibly with different lengths. Only one-dimensional samples are accepted.

03. 가설 검정 기법 - 독립표본 T-검정

(2023년 11월 기준)

◆ 주요 메서드 - 독립표본 t-검정

- ✓ 데이터의 각 변수들은 정규분포를 만족하며, 두 그룹은 등분산을 만족한다!

```
1 : from scipy import stats
2 : t_score, p_value = stats.ttest_ind(df.loc[df['supp'] == "VC", 'len'],
                                     df.loc[df['supp'] == "OJ", 'len'], equal_var = True)
3 : print(round(t_score, 4), round(p_value, 2))
    -1.9153 0.06
```

◆ 메서드 설명

- ✓ 이 검정은 두 독립 샘플의 평균(예상) 값이 동일하다는 귀무가설에 대한 검정.
- ✓ 두 집단의 분산이 동일하다고 가정함.

메서드	매개변수	설명
ttest_ind(a, b, equal_var)	a, b	데이터의 크기가 동일해야 함.
	equal_var	If true, independent t-test / If false, Welch's t-test

◆ 가설 설정

- ✓ 실험단위를 동질적인 쌍으로 묶은 다음, 각 쌍에서 관측값의 차를 이용하여 두 모평균의 차이에 관한 추론
- ✓ 실험 이전의 집단과 실험 이후의 집단이 동일한 경우 사용하는 검정(쌍체비교)이라고 한다.
- ✓ $(X_1, Y_1), (X_2, Y_2), (X_3, Y_3), \dots, (X_n, Y_n)$
- ✓ 각 쌍의 차이를 $D_i = X_i - Y_i (i = 1, 2, \dots, n)$ 로 정의할 때, D_i 는 $N(\mu_D, \sigma_D^2)$ 으로부터의 확률표본으로 가정한다.
- ✓ $\mu_D = \mu_1 - \mu_2$

가설	표현식	설명
H_0	$\mu_D = 0$	실험전후 평균의 차이는 0이다
H_1	$\mu_D \neq 0$	실험전후 평균의 차이는 0이 아니다

◆ 가정(Assumptions) 확인

- ✓ 독립성
- ✓ 정규성

04. 가설 검정 기법 - 대응표본 T-검정

(2023년 11월 기준)

◆ 정규성 검정

- ✓ 실험전후 두 변수의 차이가 정규분포를 따르는지 확인, $N \leq 30$ 이하일 때, 검정해야 함
- ✓ 각 그룹의 변수가 정규분포를 따르는지는 검정할 필요가 없음
- ✓ 각 그룹의 표본수가 모두 $N \geq 30$ 이상일 때, 중심극한정리에 의해 정규성 가정을 만족했다고 봄

가설	설명
H_0	각 자료는 정규분포를 따른다.
H_1	각 자료는 정규분포를 따르지 않는다.

◆ 확인 방법

- ✓ Shapiro-Wilk tests

◆ 가정 위반 시, Wilcoxon Signed-Ranks Test를 진행

◆ 가설 설정

- ✓ 실험단위를 동질적인 쌍으로 묶은 다음, 각 쌍에서 관측값의 차를 이용하여 두 모평균의 차이에 관한 추론
- ✓ 실험 이전의 집단과 실험 이후의 집단이 동일한 경우 사용하는 검정(쌍체비교)이라고 한다.
- ✓ $(X_1, Y_1), (X_2, Y_2), (X_3, Y_3), \dots, (X_n, Y_n)$
- ✓ 각 쌍의 차이를 $D_i = X_i - Y_i (i = 1, 2, \dots, n)$ 로 정의할 때, D_i 는 $N(\mu_D, \sigma_D^2)$ 으로부터의 확률표본으로 가정한다.
- ✓ $\mu_D = \mu_1 - \mu_2$

가설	표현식	설명
H_0	$\mu_D = 0$	실험전후 평균의 차이는 0이다
H_1	$\mu_D \neq 0$	실험전후 평균의 차이는 0이 아니다

◆ 검정통계량

- ✓ 실험 전 표본평균 X , 실험 후 표본평균 Y 의 차이의 평균(D)에 근거하여 구성

$$t = \frac{D}{s_D / \sqrt{n}} \quad S_D: X \text{와 } Y \text{의 차이에 관한 표준편차}$$

04. 가설 검정 기법 - 대응표본 T-검정

(2023년 11월 기준)

◆ 주요 메서드 - 대응표본 t-검정

- ✓ 실험 전후의 차이를 나타내는 데이터는 정규분포를 이룬다.

```
1 : from scipy import stats
2 : t_score, p_value = stats.ttest_rel(df['before_spr'], df['after_spr'])
3 : print(round(t_score, 4), round(p_value, 2))
14.8933 0.0
```

◆ 메서드 설명

- ✓ 이 검정은 두 독립 샘플의 평균(예상) 값이 동일하다는 귀무가설에 대한 검정.
- ✓ 두 집단의 분산이 동일하다고 가정함.

메서드	매개변수	설명
<code>ttest_rel(a, b)</code>	<code>a, b :</code> <code>array_like</code>	데이터의 크기가 동일해야 함.
	<code>alternative</code>	<code>option : two-sided(default), less, greather</code>

05. 적합도 검정(goodness-of-fit) : χ^2 검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 어떤 실험에서 관측도수가 가정하는 이론상의 분포를 잘 따른다는 귀무가설을 검정하는 것
- ✓ 관측도수란, 실제 실험에서 단일 특성에 의해 분류된 각 범주의 관측값을 말함
- ✓ 관측도수가 얼마나 이론상의 분포 또는 주어진 형태를 잘 따르는지를 검정하는 가설 검정 기법을 적합도 검정
- ✓ 예) 우리나라 성인의 키가 평균 μ 와 분산 σ^2 을 갖는 정규분포 $N(\mu, \sigma^2)$ 을 따른다고 가정했을 때, 이 가정이 적합한지 검정

가설	설명
H_0	성인의 키는 정규분포 $N(\mu, \sigma^2)$ 를 따른다.
H_1	성인의 키는 정규분포 $N(\mu, \sigma^2)$ 를 따르지 않는다.

◆ 검정통계량

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

O_i : 관측값
 E_i : 기대값

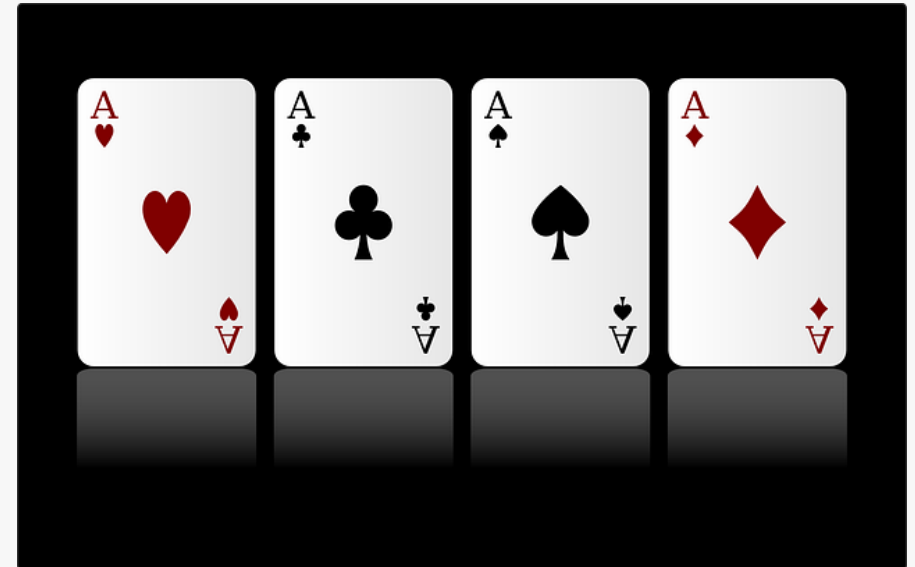
05. 적합도 검정(goodness-of-fit) : χ^2 검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 어떤 실험에서 관측도수가 가정하는 이론상의 분포를 잘 따른다는 귀무가설을 검정하는 것
- ✓ 관측도수란, 실제 실험에서 단일 특성에 의해 분류된 각 범주의 관측값을 말함
- ✓ 관측도수가 얼마나 이론상의 분포 또는 주어진 형태를 잘 따르는지를 검정하는 가설 검정 기법을 적합도 검정
- ✓ 예) 우리나라 성인의 키가 평균 μ 와 분산 σ^2 을 갖는 정규분포 $N(\mu, \sigma^2)$ 을 따른다고 가정했을 때, 이 가정이 적합한지 검정
- ✓ 예) 트럼프카드의 확률게임

Label	Index i	math.symbol	the value
hearts ♥	0	O_1	64
diamonds ♦	1	O_2	51
spades ♠	2	O_3	50
clubs ♣	3	O_4	35



<출처> <https://ethanweed.github.io/pythonbook/05.01-chisquare.html>

05. 적합도 검정(goodness-of-fit) : χ^2 검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 어떤 실험에서 관측도수가 가정하는 이론상의 분포를 잘 따른다는 귀무가설을 검정하는 것
- ✓ 관측도수란, 실제 실험에서 단일 특성에 의해 분류된 각 범주의 관측값을 말함
- ✓ 관측도수가 얼마나 이론상의 분포 또는 주어진 형태를 잘 따르는지를 검정하는 가설 검정 기법을 적합도 검정
- ✓ 예) 우리나라 성인의 키가 평균 μ 와 분산 σ^2 을 갖는 정규분포 $N(\mu, \sigma^2)$ 을 따른다고 가정했을 때, 이 가정이 적합한지 검정
- ✓ 예) 트럼프카드의 확률게임

가설	표현식	설명
H_0	$P = (.25, .25, .25, .25)$	모든 경우의 수는 동일하게 같은 확률값으로 선택하게 된다.
H_1	$P \neq (.25, .25, .25, .25)$	적어도 하나는 같은 확률값으로 선택하는 건 아니다

05. 적합도 검정(goodness-of-fit) : χ^2 검정

(2023년 11월 기준)

◆ 검정통계량

✓ 예) 트럼프카드의 확률게임

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

O_i : 관측값

E_i : 기대값

Label	Index i	math	관측값	기대 확률값	기대 빈도(E_i) (N=200)	차이값	제곱값	χ^2 통계량
hearts ♡	0	O_1	64	.25	50	14	196	196/50 = 3.92
diamonds ◇	1	O_2	51	.25	50	1	1	1/50 = 0.02
spades ♠	2	O_3	50	.25	50	0	0	0/50 = 0.00
clubs ♣	3	O_4	35	.25	50	-15	225	225/50 = 4.50

8.44

05. 적합도 검정(goodness-of-fit) : χ^2 검정

(2023년 11월 기준)

◆ 주요 메서드 - 카이제곱 검정

- ✓ 기존 분포와 동일한지 검정

```
1 : from scipy import stats
2 : f_score, p_value = stats.chisquare(observed, f_exp = expected)
3 : print(round(t_score, 4), round(p_value, 2))
    16.73 0.0
```

◆ 메서드 설명

- ✓ 카이제곱 검정은 범주형 데이터에 주어진 빈도가 있다는 귀무가설을 테스트.

메서드	매개변수	설명
<code>chisquare(f_obs, f_exp)</code>	f_obs : array_like	각 category에서 관측된 빈도수
	f_exp : array_like	각 category의 비율에 따른 기대 빈도수

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 가설 설정

- ✓ 두 범주형 변수 또는 특성이 존재할 때 두 특성이 서로 독립인지 여부에 대해 알아보는 검정
- ✓ 한 특성이 다른 특성에 영향을 미치는지 여부에 대하여 알아보는 검정
- ✓ 예) 영화 장르별 간식 구매의 상관성 검정

영화 장르	간식류 구매	간식류 비구매	행의 합계
작업	50	75	125
코미디	125	175	300
가족	90	30	120
공포	45	10	55
열의 합계	310	290	전체 합계 = 600

<출처 : https://www.jmp.com/ko_kr/statistics-knowledge-portal/chi-square-test/chi-square-test-of-independence.html>

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 가설 검정

가설	설명
H_0	영화 장르와 간식류 구입은 서로 독립적이다.
H_1	영화 장르와 간식류 구입은 서로 독립적인 것은 아니다 (= 상관성이 존재한다)

◆ 검정통계량

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(E_{ij} - O_{ij})^2}{E_{ij}}$$

$$df = (r - 1)(c - 1)$$

◆ 카이제곱 분포표

$$\checkmark \alpha = 0.05, df = (r - 1)(c - 1)$$

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

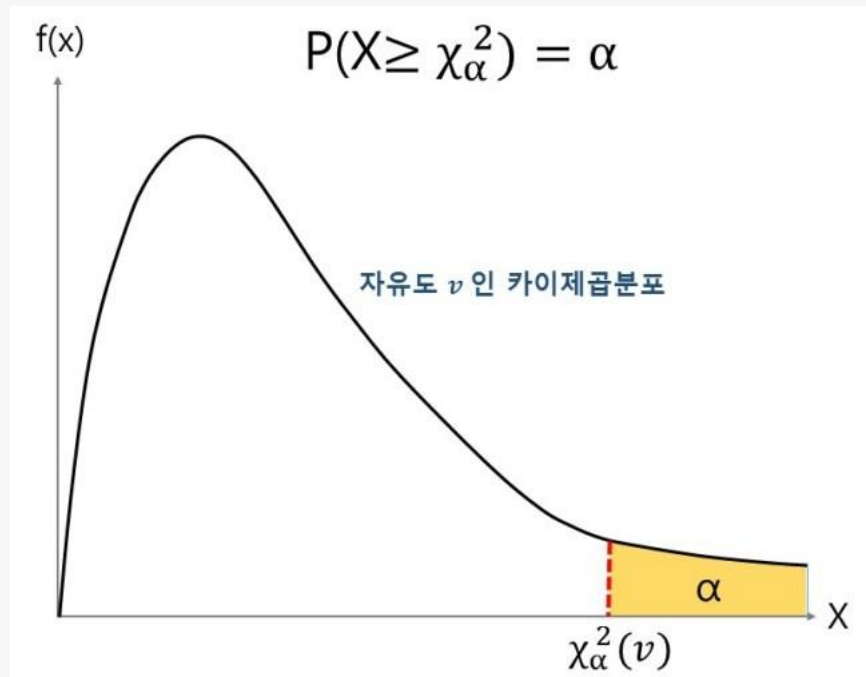
◆ 카이제곱 분포표

✓ $\alpha = 0.05, df = (r - 1)(c - 1)$

◆ χ^2 와 카이제곱 분포표와 비교

✓ $\chi^2 > \text{카이제곱 분포표} : H_0 \text{ 기각}$

✓ $\chi^2 < \text{카이제곱 분포표} : H_0 \text{ 채택}$



Degree of Freedom	Probability of Exceeding the Critical Value								
	0.99	0.95	0.90	0.75	0.50	0.25	0.10	0.05	0.01
1	0.000	0.004	0.016	0.102	0.455	1.32	2.71	3.84	6.63
2	0.020	0.103	0.211	0.575	1.386	2.77	4.61	5.99	9.21
3	0.115	0.352	0.584	1.212	2.366	4.11	6.25	7.81	11.34
4	0.297	0.711	1.064	1.923	3.357	5.39	7.78	9.49	13.28
5	0.554	1.145	1.610	2.675	4.351	6.63	9.24	11.07	15.09
6	0.872	1.635	2.204	3.455	5.348	7.84	10.64	12.59	16.81
7	1.239	2.167	2.833	4.255	6.346	9.04	12.02	14.07	18.48
8	1.647	2.733	3.490	5.071	7.344	10.22	13.36	15.51	20.09
9	2.088	3.325	4.168	5.899	8.343	11.39	14.68	16.92	21.67
10	2.558	3.940	4.865	6.737	9.342	12.55	15.99	18.31	23.21
11	3.053	4.575	5.578	7.584	10.341	13.70	17.28	19.68	24.72
12	3.571	5.226	6.304	8.438	11.340	14.85	18.55	21.03	26.22
13	4.107	5.892	7.042	9.299	12.340	15.98	19.81	22.36	27.69
14	4.660	6.571	7.790	10.165	13.339	17.12	21.06	23.68	29.14
15	5.229	7.261	8.547	11.037	14.339	18.25	22.31	25.00	30.58
16	5.812	7.962	9.312	11.912	15.338	19.37	23.54	26.30	32.00
17	6.408	8.672	10.085	12.792	16.338	20.49	24.77	27.59	33.41
18	7.015	9.390	10.865	13.675	17.338	21.60	25.99	28.87	34.80
19	7.633	10.117	11.651	14.562	18.338	22.72	27.20	30.14	36.19
20	8.260	10.851	12.443	15.452	19.337	23.83	28.41	31.41	37.57
22	9.542	12.338	14.041	17.240	21.337	26.04	30.81	33.92	40.29
24	10.856	13.848	15.659	19.037	23.337	28.24	33.20	36.42	42.98
26	12.198	15.379	17.292	20.843	25.336	30.43	35.56	38.89	45.64
28	13.565	16.928	18.939	22.657	27.336	32.62	37.92	41.34	48.28
30	14.953	18.493	20.599	24.478	29.336	34.80	40.26	43.77	50.89
40	22.164	26.509	29.051	33.660	39.335	45.62	51.80	55.76	63.69
50	27.707	34.764	37.689	42.942	49.335	56.33	63.17	67.50	76.15
60	37.485	43.188	46.459	52.294	59.335	66.98	74.40	79.08	88.38
Not Significant								Significant	

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 각 셀의 기대도수 계산

영화 장르	간식류 구매	간식류 비구매	행의 합계
작업	50 $125 \times 310/600 = 64.58$	75 $125 \times 290/600 = 60.42$	125
코미디	125 155	175 145	300
가족	90 62	30 58	120
공포	45 28.42	10 26.58	55
열의 합계	310	290	전체 합계 = 600

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 각 셀의 **검정통계량** 계산

영화 장르	간식류 구매	간식류 비구매	행의 합계
작업	차이 $50 - 64.58 = -14.58$ 제곱값 : 212.67 기대값으로 나누기 : $212.67 / 64.58 = 3.29$	3.52	125
코미디	5.81	6.21	300
가족	12.65	13.52	120
공포	9.68	10.35	55
열의 합계	310	290	전체 합계 = 600
검정통계량	$3.29 + 3.52 + 5.81 + 6.21 + 12.65 + 13.52 + 9.68 + 10.35 = 65.03$		
자유도	$(r - 1) \times (c - 1) = (4 - 1) \times (2 - 1) = 3$		

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 각 셀의 **검정통계량** 계산

영화 장르	간식류 구매	간식류 비구매	행의 합계
작업	3.29	3.52	125
코미디	5.81	6.21	300
가족	12.65	13.52	120
공포	9.68	10.35	55
열의 합계	310	290	전체 합계 = 600
검정통계량	$3.29 + 3.52 + 5.81 + 6.21 + 12.65 + 13.52 + 9.68 + 10.35 = 65.03$		
자유도	$(r - 1) \times (c - 1) = (4 - 1) \times (3 - 1) = 3$		
비교	검정통계량 (65.03) > 카이제곱 분포표 7.81 ($\alpha = 0.05$, $df = 3$)		
해석	H0 기각 & H1 채택, 영화장르와 간식류 구매사이에 상관성이 존재한다.		

06. 독립성 검정 : χ^2 검정

(2023년 11월 기준)

◆ 주요 메서드 - 카이제곱 검정

- ✓ 두 범주형 또는 명목형 변수가 서로 연관이 있는 지 가능성 여부를 확인하는 통계 가설 검정

```
1 : from scipy import stats
2 : result = stats.chi2_contingency([X1, X2, X3])
3 : f_score, p_value = result[0], result[1]
4 : print(round(f_score, 4), round(p_value, 4))

30.1275 0.0
```

◆ 메서드 설명

- ✓ 분할표(contingency table)에서 변수의 독립성에 대한 카이제곱 검정.

메서드	매개변수	설명
<code>chi2_contingency(observed)</code>	<code>observed</code> : <code>array_like</code>	분할표를 말하며, 표에는 각 범주에서 관찰된 빈도(즉, 발생 횟수)가 포함되어 있습니다. 2차원의 경우, 이 테이블은 종종 "R x C 테이블"로 설명됨

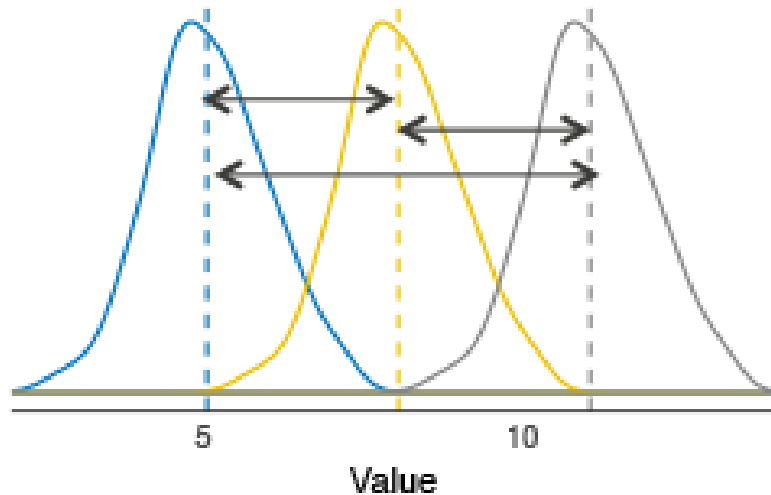
07. 분산분석

(2023년 11월 기준)

- ◆ 두개 이상의 집단에서 그룹 평균 간 차이를 그룹 내 변동에 비교하여 살펴보는 통계분석 방법
 - ✓ 집단 내에 분산보다 다른 집단과의 분산이 더 크면 유의하다고 할 수 있다는 개념을 가지고 있음

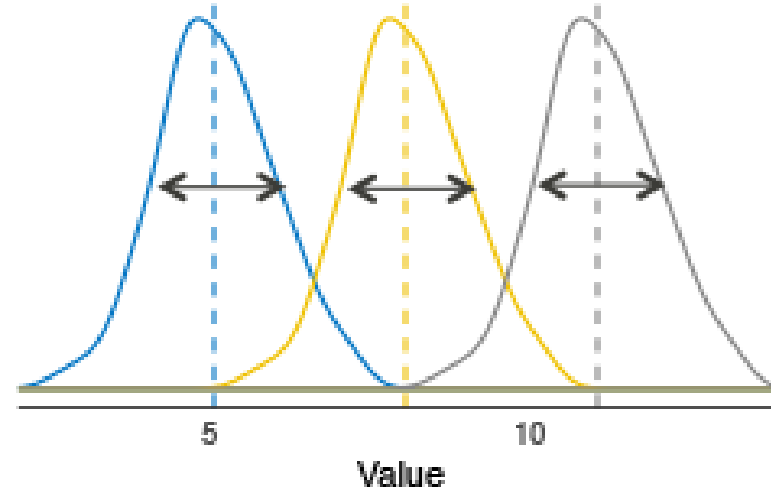
A

Between-group variation
(i.e. Differences among group means)



B

Within-group variation
(i.e. Variability within each group)



- ◆ 두개 이상의 집단에서 그룹 평균 간 차이를 그룹 내 변동에 비교하여 살펴보는 통계분석 방법
 - ✓ 집단 내에 분산보다 다른 집단과의 분산이 더 크면 유의하다고 할 수 있다는 개념을 가지고 있음

구분	명칭	독립변수 개수	종속변수 개수
단일변량 분산분석	일원배치 분산분석	1개	1개
	이원배치 분산분석	2개	
	다원배치 분산분석	3개 이상	
다변량 분산분석	MANOVA	1개 이상	2개 이상

- ◆ 일원배치분산분석 (One-Way ANOVA)
 - ✓ 반응값에 대한 하나의 범주형 변수의 영향을 알아보기 위해 사용되는 검증 방법
 - ✓ 모집단의 수에는 제한이 없으며, 각 표본의 수는 같지 않아도 된다.
 - ✓ F 검정 통계량을 이용한다.

요인	제곱합	자유도	평균제곱	F-value	P
집단 간	SSB	집단수 - 1	$\frac{\text{집단 간 제곱합}}{\text{자유도}}$	$\frac{\text{집단 간 평균제곱}}{\text{집단 내 평균제곱}}$	-
집단 내	SSW	자료 수 - 집단 수	$\frac{\text{집단 내 제곱합}}{\text{자유도}}$		-
합계	SST	자료 수 - 1	-		-

- ◆ 용어
 - ✓ SST(Sum of Squares) : 총 변동
 - ✓ SSB(Sum of Squares Between) : 집단 간 변동
 - ✓ SSW(Sum of Squares Within) : 집단 내 변동

◆ 일원배치분산분석 (One-Way ANOVA)

- ✓ 반응값에 대한 하나의 범주형 변수의 영향을 알아보기 위해 사용되는 검증 방법
- ✓ 모집단의 수에는 제한이 없으며, 각 표본의 수는 같지 않아도 된다.
- ✓ F 검정 통계량을 이용한다.

◆ 가정

- ✓ 집단의 측정치는 서로 독립적이어야 한다.
- ✓ 각 집단의 데이터를 정규분포를 따른다.
 - 집단별 크기가 25미만이면, 정규성을 체크함
 - 만약, 정규성 가정 위반시, Kruskal-Wallis Test를 사용함
- ✓ 표본 크기가 매우 다를 때, 분산의 동질성을 확인하며, Levene Test를 사용
 - 만약, 등분산 가정이 위반되면 Welch Test를 사용함

◆ 일원배치분산분석 (One-Way ANOVA)

- ✓ 반응값에 대한 하나의 범주형 변수의 영향을 알아보기 위해 사용되는 검증 방법
- ✓ 모집단의 수에는 제한이 없으며, 각 표본의 수는 같지 않아도 된다.
- ✓ F 검정 통계량을 이용한다.

◆ 가설

- ✓ 귀무가설 : K개의 집단 간 평균에는 차이가 없다.
- ✓ 대립가설 : K개의 집단 간 평균이 모두 같다고 할 수 없다.

◆ 주요 메서드 - 일원배치분산분석 (One Way ANOVA)

- ✓ 일원 분산 분석은 두 개 이상의 그룹이 동일한 모집단 평균을 가지고 있다는 귀무가설을 테스트.
- ✓ 테스트는 크기가 서로 다른 두 개 이상의 그룹의 샘플에 적용됩니다.

```
1 : from scipy import stats
2 : f_score, p_value = stats.f_oneway(X1, X2, X3)
3 : print(round(f_score, 4), round(p_value, 4))
4 : 49.16 0.0
```

◆ 메서드 설명

- ✓ 일원배치분산분석을 수행하며, 각 sample은 Series 형태, array 형태로 오면 된다.

메서드	매개변수	설명
<code>f_oneway(sample1, ...)</code>	sample1 : array_like	The sample measurements for each group.

◆ 주요 메서드 - 일원배치분산분석 (One Way ANOVA)

- ✓ 일원 분산 분석은 두 개 이상의 그룹이 동일한 모집단 평균을 가지고 있다는 귀무가설을 테스트.
- ✓ 테스트는 크기가 서로 다른 두 개 이상의 그룹의 샘플에 적용됩니다.
- ✓ 귀무가설 기각, 대립가설 채택 시, 사후검정을 진행해야 함

◆ 사후검정 파이썬 코드 예시

```
1 : from statsmodels.stats.multicomp import pairwise_tukeyhsd
2 : posthoc = pairwise_tukeyhsd(df['outcome_variables'], df['group'])
3 : print(posthoc.summary())
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1      group2      meandiff p-adj      lower      upper      reject
-----
setosa versicolor -0.658      0.0 -0.8189 -0.4971      True
setosa  virginica -0.454      0.0 -0.6149 -0.2931      True
versicolor virginica 0.204 0.0088  0.0431  0.3649      True
-----
```

◆ 모수 통계 가설 검정

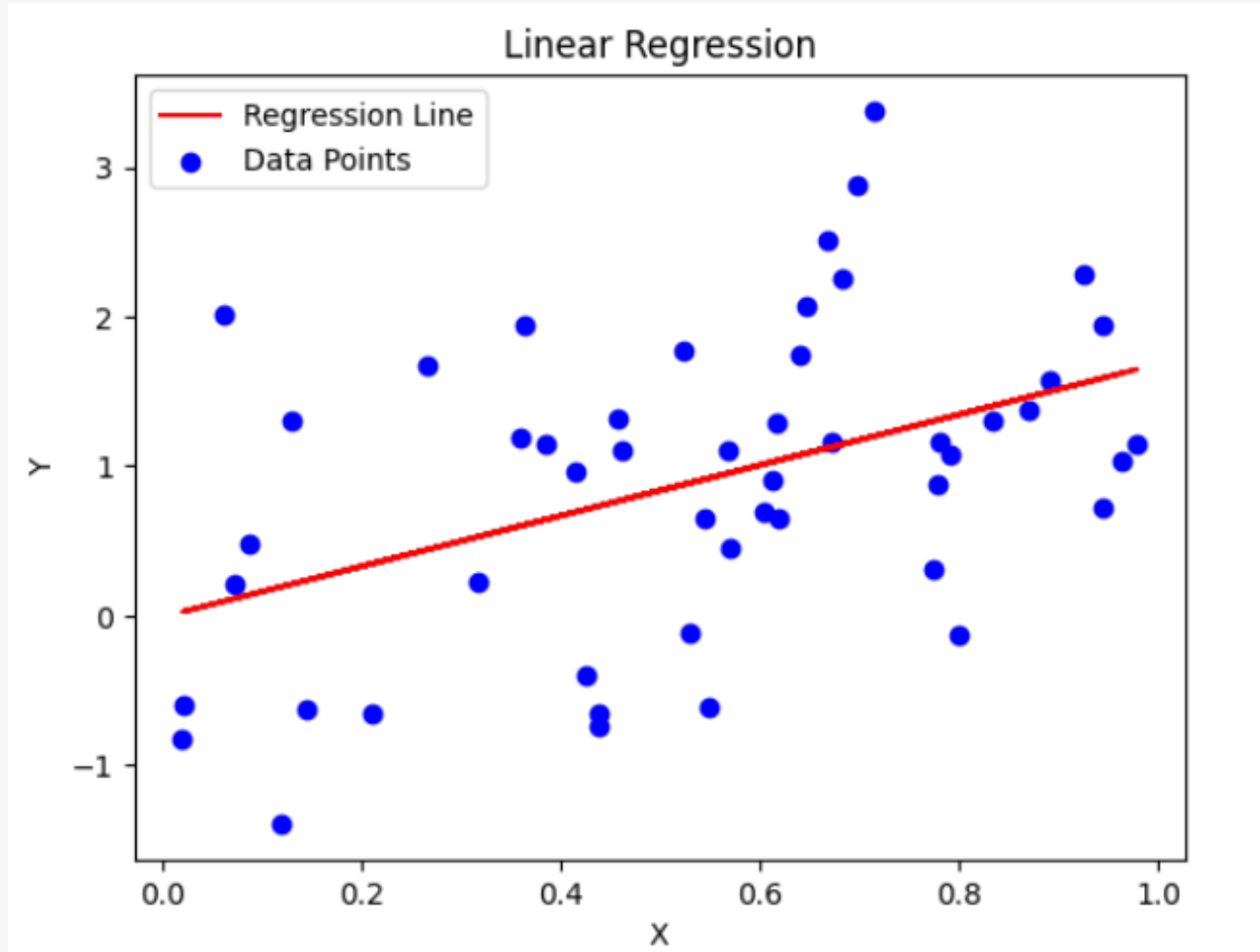
검정 방법	메서드	라이브러리 링크
단일표본 T-검정	.ttest_1samp()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_1samp.html
독립표본 T-검정	.ttest_ind()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_ind.html
대응표본 T-검정	.ttest_rel()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_rel.html
일원분산분석	.f_oneway()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.f_oneway.html
카이제곱검정(적합도 검정)	.chisquare()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chisquare.html
카이제곱검정(독립성 검정)	.chi2_contingency()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chi2_contingency.html

◆ 모수 통계 가설 검정

검정 방법	메서드	라이브러리 링크
정규성 검정	.shapiro()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html
등분산성 검정	.levene()	https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.levene.html

◆ 기본 개념

- ✓ 일반적으로 하나의 종속변수 Y 와 여러 독립(=예측)변수 간의 관계를 모델링하는데 사용됨



◆ 선형 회귀 모형 진단

- ✓ 회귀 모형이 적합하려면, 4가지 가정을 모두 만족시켜야 함
- ✓ 예측값과 실제 값의 차이인 잔차(Residual)를 이용하여 검증

가정	의미	진단 방법
선형성	종속변수는 독립변수의 선형 함수	잔차 산점도 : 선형성 확인
독립성	독립변수 사이에는 상관관계가 없어야 함	잔차 산점도 : 특정한 경향성이 없어야 함 더빈-왓슨 검정(Durbin-Watson Test)
등분산성	오차항의 분산은 등분산임	잔차 산점도 : 고르게 분포되어야 함
정규성	오차항의 평균은 0임	Shapiro Wilk 검정 Kolmogorov-Smirnov Goodness of Fit Test Q-Q plot

◆ 검정 설명

- ✓ Shapiro Wil 검정 : 데이터의 정규성을 검증하는 방법
- ✓ Kolmogorov-Smirnov 검정 : 데이터가 예상되는 분포에 얼마나 잘 맞는지를 검정

10. 표준편차 vs. 표준오차

(2023년 11월 기준)

◆ 비교

표준편차(Standard Deviation)	표준오차(Standard Error)
$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$ 표본평균	$S.E = \frac{\sigma}{\sqrt{n}}$ $S.E = \frac{s}{\sqrt{n}}$
✓ 각 데이터가 평균과 얼마나 차이를 가지는지 알려주는 지표	✓ 표본평균의 표준편차 ✓ 추정값인 표본평균들의 참값인 모평균과의 표준적인 차이 ✓ 수식에서 n 이 커지면 표준오차는 줄어듦

